AMD GCN/CDNA Architecture Training

Memory, IO, and CU Architecture on gfx9 GPUs

Joseph Greathouse

Machine Learning Software Engineering (MLSE)

RTG, AMD

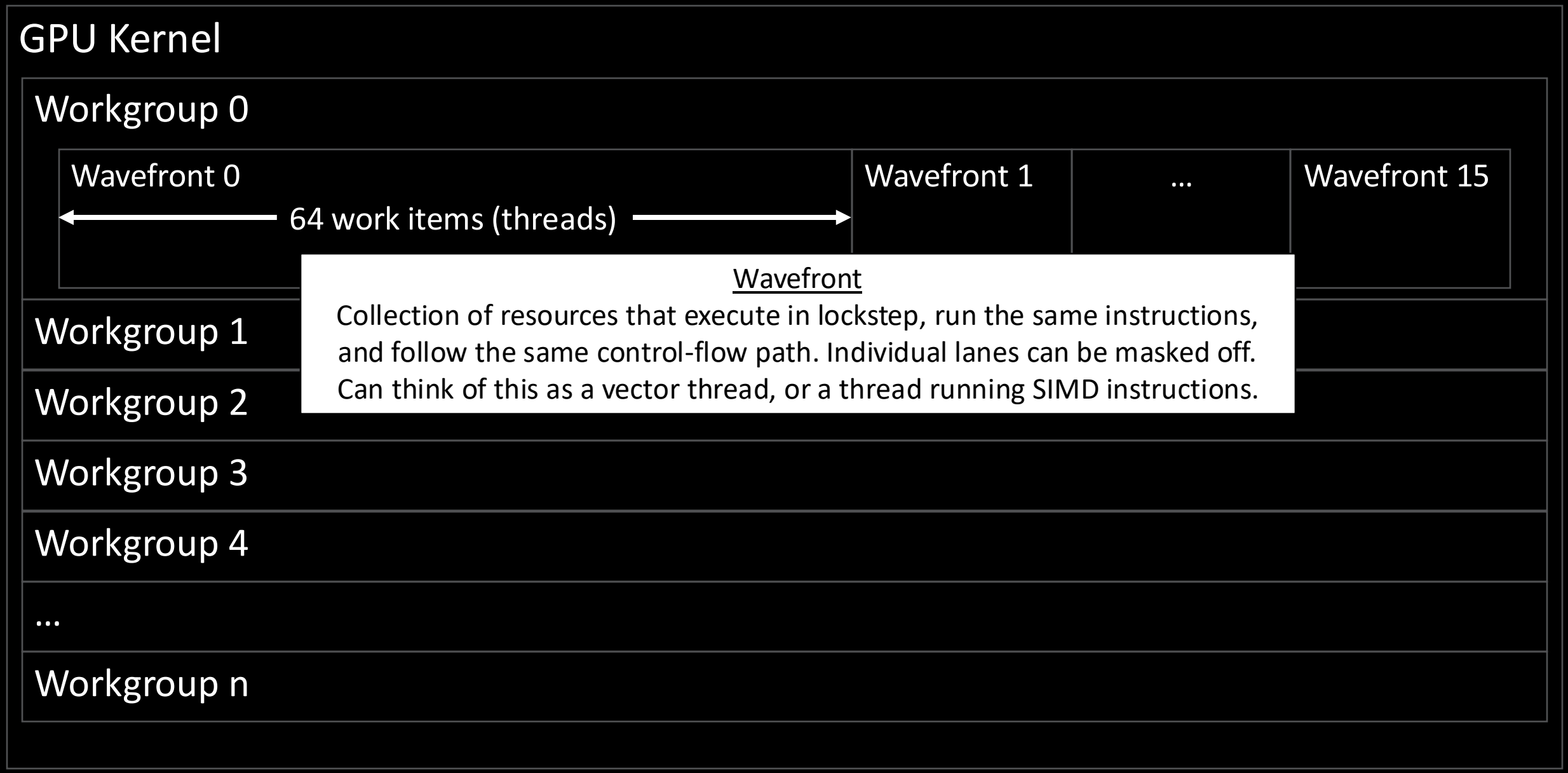
Apr. 14, 2020

AGENDA

GCN Assembly	AMD GCN Assembly Instruction Classes
	SIMT Program Example
	Branching in a SIMT Program
Hardware Internals	Compute Units
	Memory System
Input/Output	Efficiency Arbiters
	SDMAs

AMD GCN Assembly Instruction Classes

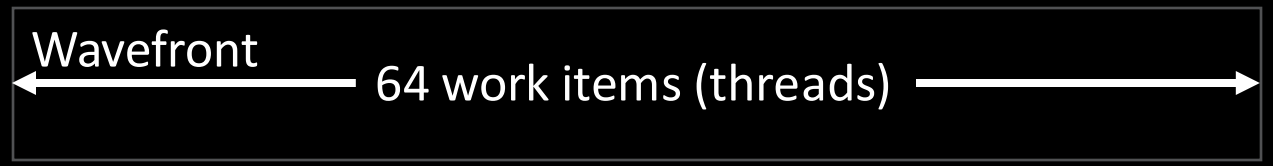
Reminder of GPU Kernel Layout



Wavefront

Collection of resources that execute in lockstep, run the same instructions, and follow the same control-flow path. Individual lanes can be masked off. Can think of this as a vector thread, or a thread running SIMD instructions.

Wavefronts on AMD GCN GPUs



- AMD GCN Compute Units execute wavefronts, not work items.
- Each wavefront has instructions in various classes:
  - Vector ALU (VALU): 64-wide ALU instruction.
    - FP16, FP32, FP64, integer, bitwise operators can all be VALU.
    - Each lane can operate on a different value. Individual lanes can be masked off.
  - Vector Memory (VMEM): 64-wide memory instruction.
    - Each lane can touch a different location. Individual lanes can be masked off.
  - Local Data Share (LDS): 64-wide memory operations to per-CU scratchpad.
    - Each lane can touch a different location. Individual lanes can be masked off.
  - Scalar ALU (SALU): ALU operation if every thread is working on identical data. Has its own registers.
  - Scalar Memory (SMEM): Identical memory operation in all lanes. Uses scalar register file.
  - Branch: Used if the entire wavefront wants to branch.
    - Per-thread branching will run both sides of the branch with different lane masks.
  - Internal: Wavefront bookkeeping. NOPs, workgroup barriers, wait-for-memory, etc.

An Example SIMT Kernel

SIMT Shifted Copy Kernel in HIP

- Every thread copies a float from array in[] to array out[]

```
__global__ void shifted_copy (float *in, float *out) {
  size_t gid = blockDim.x * blockIdx.x + threadIdx.x
  out[gid] = in[gid+4];
}
```

- SIMT programming model implies parallelism between threads.
- Every HIP Thread reads one value and writes one value, based on its global thread ID

SIMT Copy Kernel in AMD GCN Assembly

```
__global__ void shifted_copy (float *in, float *out) {
  size_t gid = blockDim.x * blockIdx.x + threadIdx.x
  out[gid] = in[gid + 4];
}
```

- Skipping some function prologue. Starting state:
  - Scalar registers SGPR5 and SGPR4 concatenate together to hold pointer to kernel argument structure.
  - Vector registers VGPR3 and VGPR2 concatenated together hold the per-thread global ID variable “gid”

```
s_load_dwordx4 s[0:3], s[4:5], 0 ; Load in[] to SGPR1:SGPR0 and out[] into SGPR3:SGPR2.
v_lshlrev_b64 v[2:3], 2, v[2:3] ; GID*=4 for float array offset.
s_waitcnt lgkmcnt(0) ; Wavefront does not proceed until previous SMEM access completes.
s_add_u32 s0, 16, s0 ; Add 16 to SGPR0 to shift in[] four floats over. May cause carry-out.
s_addc_u32 s1, 0, s1 ; Add any carry from the previous instruction into SGPR1.
v_add_co_u32 v0, s0, v2 ; Add low GID*4 to low in[]. Store in VGPR0.
v_addc_co_u32 v1, s1, v3 ; Add high GID*4 to high in[] with carry-in. Store in VGPR1.
global_load_dword v4, v[0:1] ; Read 64 floats from 64 locations from VGPR1:VGPR0 into VGPR4.
v_add_co_u32 v0, s2, v2 ; Add low GID*4 to low out[]. Store in VGPR0.
v_addc_co_u32 v1, s3, v3 ; Add high GID*4 to high out[] with carry-in. Store in VGPR1.
s_waitcnt vmcnt(0) ; Wavefront does not proceed until all previous VMEM accesses complete.
global_store_dword v[0:1], v4 ; Store 64 floats from VPGR4 into 64 locations from VGPR1:VGPR0.
s_endpgm ; This wavefront is done. Exit when the store completes.
```

SIMT Copy Kernel in AMD GCN Assembly

```
__global__ void shifted_copy (float *in, float *out) {
  size_t gid = blockDim.x * blockIdx.x + threadIdx.x
  out[gid] = in[gid + 4];
}
```

- Skipping some function prologue. Starting state:
  - Scalar registers SGPR5 and SGPR4 concatenate together to hold pointer to kernel argument structure.
  - Vector registers VGPR3 and VGPR2 concatenated together hold the per-thread GID variable

```
s_load_dwordx4 s[0:3], s[4:5], 0
v_lshlrev_b64 v[2:3], 2, v[2:3]
s_waitcnt lgkmcnt(0)
s_add_u32 s0, 16, s0
s_addc_u32 s1, 0, s1
v_add_co_u32 v0, s0, v2
v_addc_co_u32 v1, s1, v3
global_load_dword v4, v[0:1]
v_add_co_u32 v0, s2, v2
v_addc_co_u32 v1, s3, v3
s_waitcnt vmcnt(0)
global_store_dword v[0:1], v4
s_endpgm
```

Scalar Memory

Vector ALU

Scalar ALU

Vector Memory

Internal

An Example SIMT Kernel with Branching

SIMT Shifted Copy Kernel in HIP

- Conditionally copy some values from array in[] to array out[]

```
__global__ void conditional_copy (double *in, double *out) {
  size_t gid = blockDim.x * blockIdx.x + threadIdx.x
  if (in[gid] > 0)
    out[gid] = in[gid];
}
```

- Each HIP Thread can read a different value, some will go into the conditional statement

SIMT Copy Kernel in AMD GCN Assembly

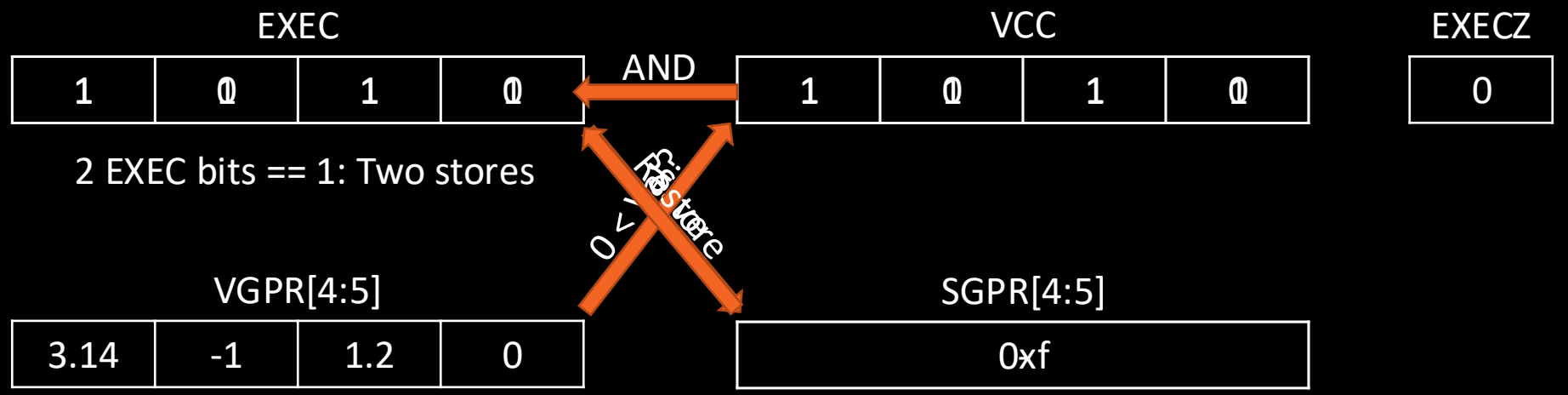
```
__global__ void conditional_copy (double *in, double *out) {
  size_t gid = blockDim.x * blockIdx.x + threadIdx.x
  if (in[gid] > 0.)
    out[gid] = in[gid];
}
```

- Skipping some function prologue. Starting state:
  - Scalar registers SGPR5 and SGPR4 concatenate together to hold pointer to kernel argument structure.
  - Vector registers VGPR3 and VGPR2 concatenated together hold the per-thread global ID variable “gid”

```
s_load_dwordx4 s[0:3], s[4:5], 0 ; Load in[] to SGPR1:SGPR0 and out[] into SGPR3:SGPR2.
v_lshlrev_b64 v[2:3], 3, v[2:3] ; GID*=8 for double array offset.
s_waitcnt lgkmcnt(0) ; Wavefront does not proceed until previous SMEM access completes.
v_add_co_u32 v0, s0, v2 ; Add low GID*8 to low in[]. Store in VGPR0.
v_addc_co_u32 v1, s1, v3 ; Add high GID*8 to high in[] with carry-in. Store in VGPR1.
global_load_dwordx2 v[4:5], v[0:1] ; Read 64 doubles from 64 locations from VGPR1:VGPR0 into VGPR5:VGPR4.
s_waitcnt vmcnt(0) ; Wavefront does not proceed until all previous VMEM accesses complete.
v_cmp_lt_f64 vcc, 0, v[4:5] ; Set this thread’s bit in VCC if 0 < VGPR[4:5] (0. < in[gid]).
s_and_saveexec_b64 s[4:5], vcc ; Save old EXEC into S[4:5]. AND VCC into the EXEC mask.
s_cbranch_execz BB0_2 ; If EXEC mask is all 0, branch to BB0_2 label.
v_add_co_u32 v0, s2, v2 ; Add low GID*8 to low out[] if this thread’s EXEC bit == 1.
v_addc_co_u32 v1, s3, v3 ; Add high GID*8 to high out[] with carry-in if EXEC bit == 1.
global_store_dword v[0:1], v[4:5] ; Store to output address if this thread’s EXEC bit == 1.
BB0_2:
s_or_b64 exec, exec, s[4:5] ; OR saved-off EXEC mask back into the current EXEC mask.
s_endpgm ; This wavefront is done. Exit when the store completes.
```

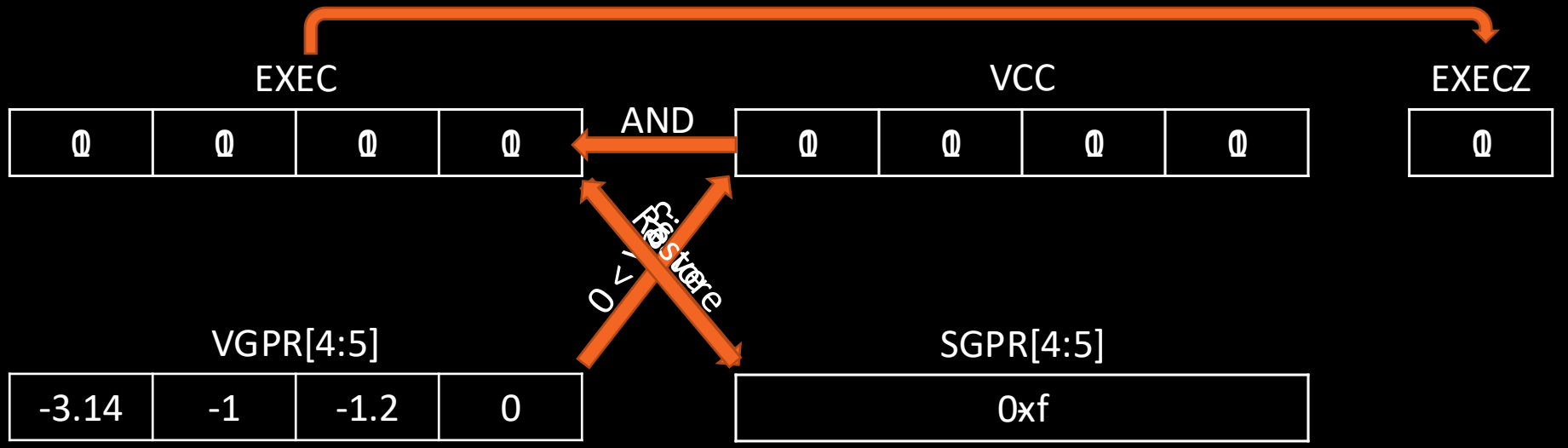
SIMT Branching Mechanisms

```
v_cmp_lt_f64 vcc, 0, v[4:5] ; Set this thread's bit in VCC if 0 < VGPR[4:5] (0. < in[gid]).
s_and_saveexec_b64 s[4:5], vcc ; Save old EXEC into S[4:5]. AND VCC into the EXEC mask.
s_cbranch_execz BB0_2 ; If EXEC mask is all 0, branch to BB0_2 label.
v_add_co_u32 v0, s2, v2 ; Add low GID*8 to low out[] if this thread's EXEC bit == 1.
v_addc_co_u32 v1, s3, v3 ; Add high GID*8 to high out[] with carry-in if EXEC bit == 1.
global_store_dwordx2 v[0:1], v[4:5] ; Store to output address if this thread's EXEC bit == 1.
BB0_2:
s_or_b64 exec, exec, s[4:5] ; OR saved-off EXEC mask back into the current EXEC mask.
```



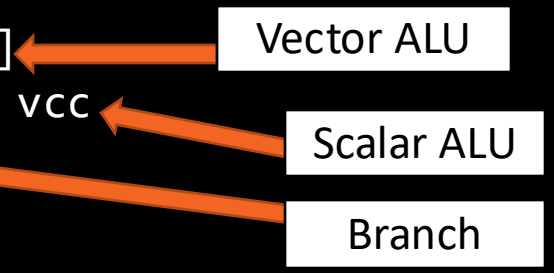
SIMT Branching Mechanisms

```
v_cmp_lt_f64 vcc, 0, v[4:5] ; Set this thread's bit in VCC if 0 < VGPR[4:5] (0. < in[gid]).
s_and_saveexec_b64 s[4:5], vcc ; Save old EXEC into S[4:5]. AND VCC into the EXEC mask.
s_cbranch_execz BB0_2 ; If EXEC mask is all 0, branch to BB0_2 label.
v_add_co_u32 v0, s2, v2 ; Add low GID*8 to low out[] if this thread's EXEC bit == 1.
v_addc_co_u32 v1, s3, v3 ; Add high GID*8 to high out[] with carry-in if EXEC bit == 1.
global_store_dwordx2 v[0:1], v[4:5] ; Store to output address if this thread's EXEC bit == 1.
BB0_2:
s_or_b64 exec, exec, s[4:5] ; OR saved-off EXEC mask back into the current EXEC mask.
```



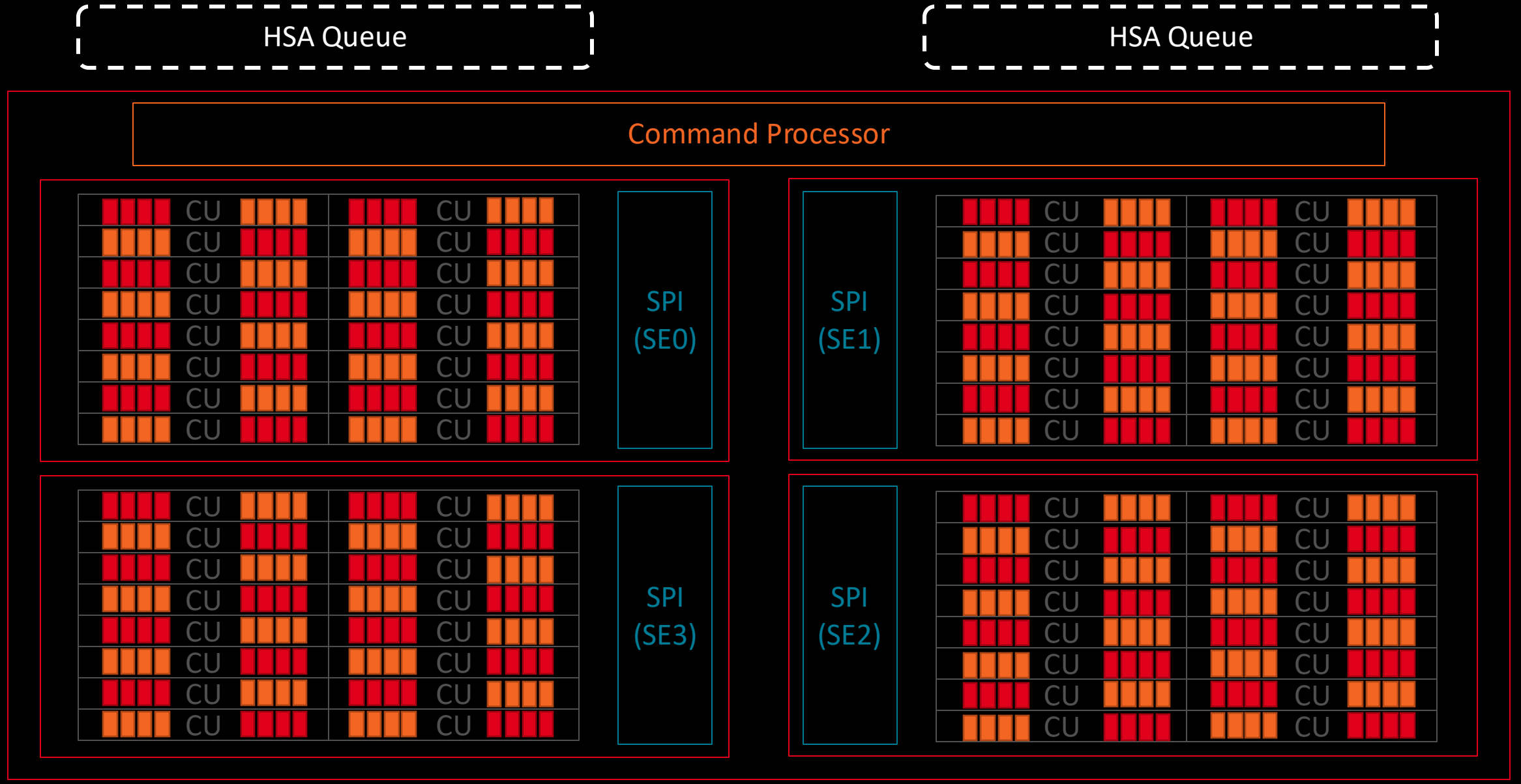
SIMT Branching Mechanisms

```
v_cmp_lt_f64 vcc, 0, v[4:5]
s_and_saveexec_b64 s[4:5], vcc
s_cbranch_execz BB0_2
v_add_co_u32 v0, s2, v2
v_addc_co_u32 v1, s3, v3
global_store_dwordx2 v[0:1], v[4:5]
BB0_2:
s_or_b64 exec, exec, s[4:5]
```

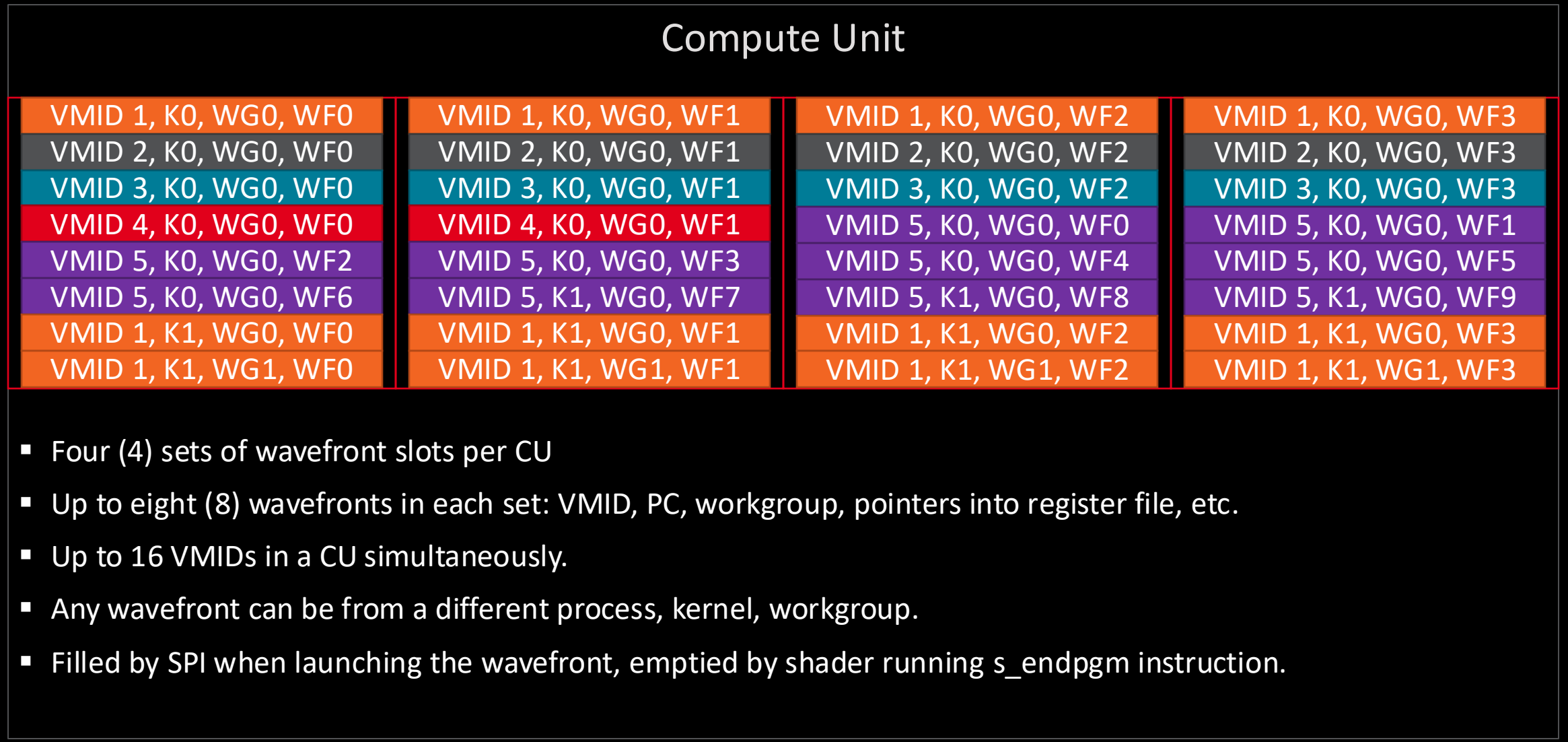


GCN Compute Unit Internals

Wavefronts in a Sea of Compute Units

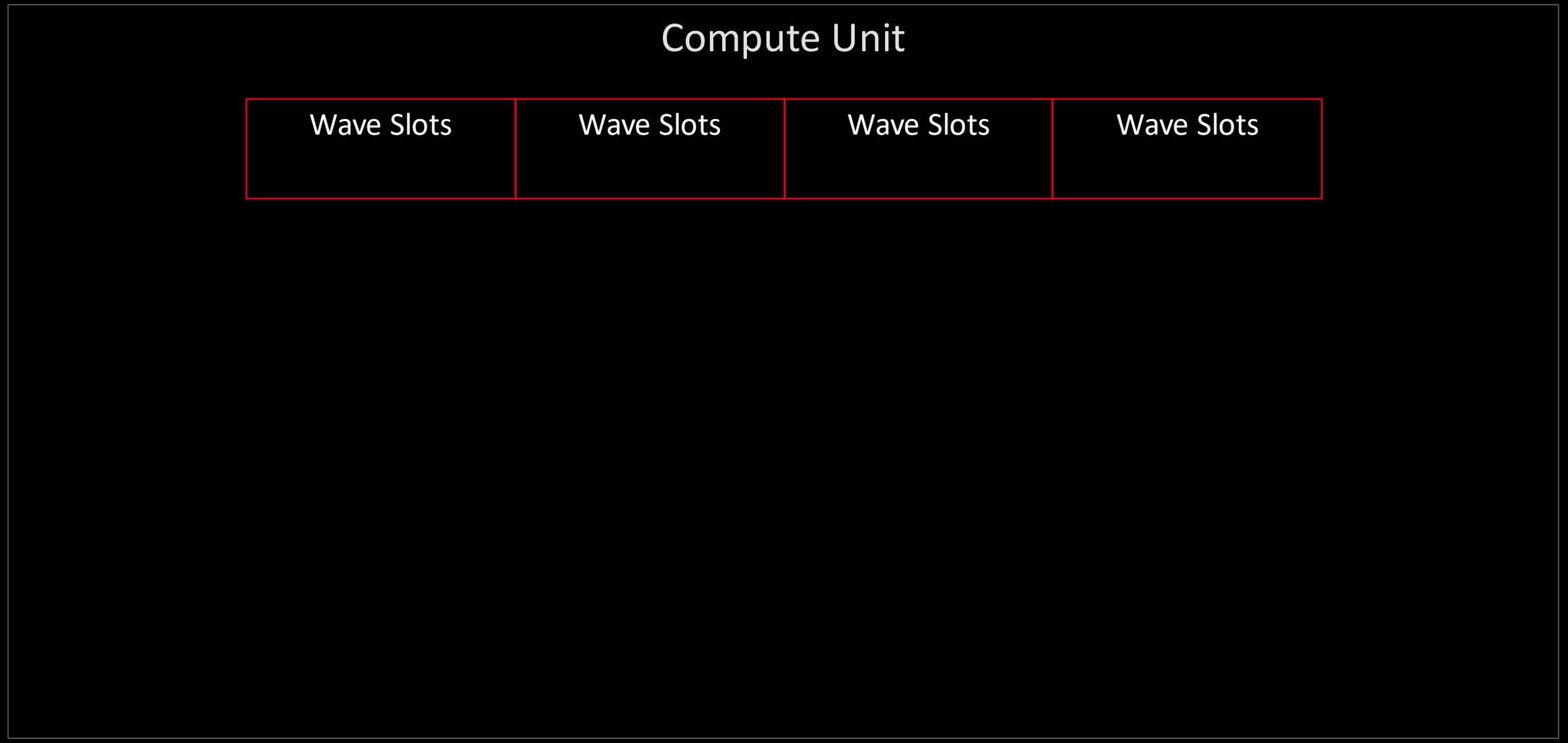


Inside a Compute Unit – Wavefront Slots



- Four (4) sets of wavefront slots per CU
- Up to eight (8) wavefronts in each set: VMID, PC, workgroup, pointers into register file, etc.
- Up to 16 VMIDs in a CU simultaneously.
- Any wavefront can be from a different process, kernel, workgroup.
- Filled by SPI when launching the wavefront, emptied by shader running s_endpgm instruction.

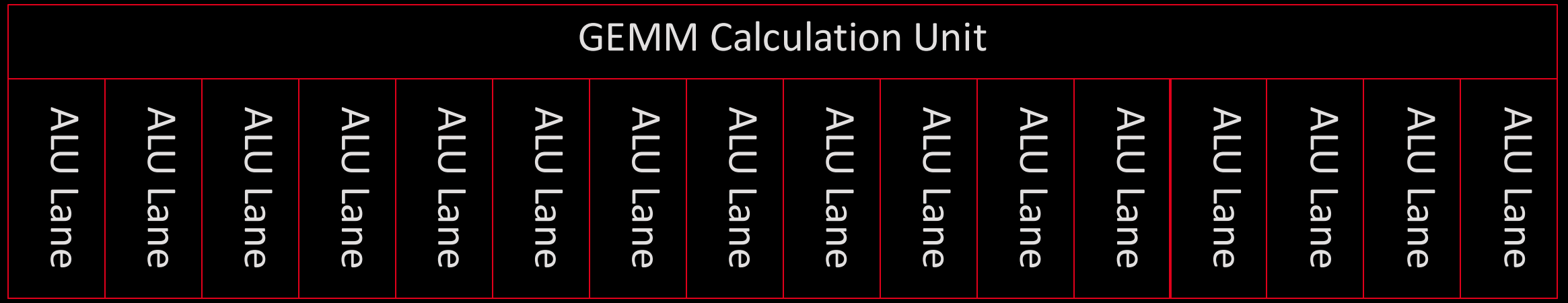
Compute Unit Internals



AMD GCN Instruction Classes

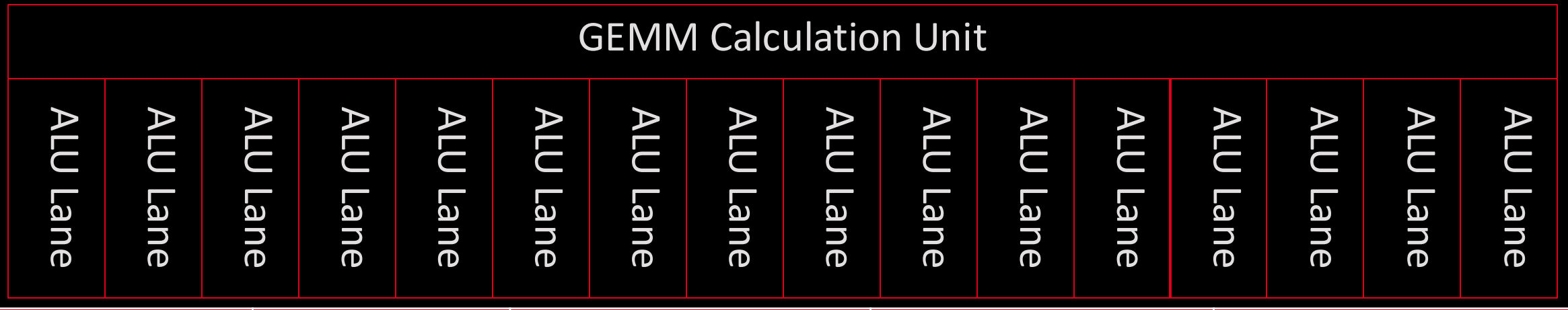
- Vector ALU (VALU): 64-wide ALU instruction.
- Vector Memory (VMEM): 64-wide memory instruction.
- Local Data Share (LDS): 64-wide memory operations to CU scratchpad.
- Scalar ALU (SALU): ALU operation if every thread is working on identical data.
- Scalar Memory (SMEM): Identical memory operation in all lanes.
- Branch: Used if the entire wavefront wants to branch.
- Internal: Wavefront bookkeeping. NOPs, workgroup barriers, wait-for-memory, etc.

Inside a Compute Unit – Vector ALUs



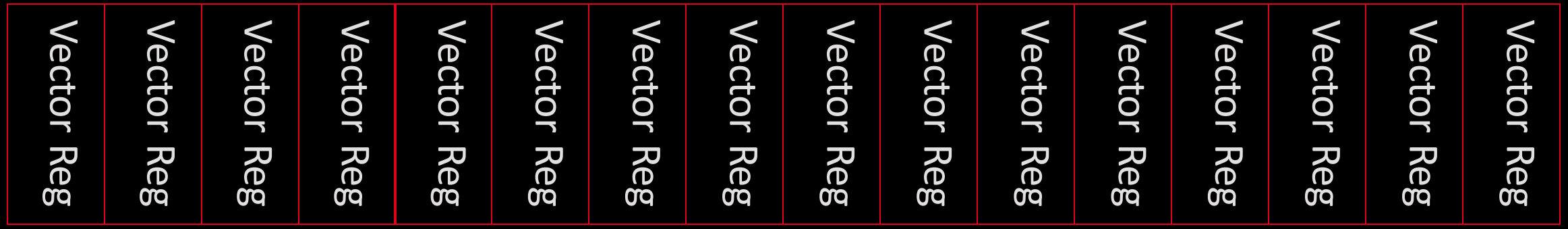
- 16-wide Vector Arithmetic Logic Unit: **SIMD-16**
  - 4 per CU. Each will only run VALU instructions from its associated set of wave slots.
- Performs vectorized logical, integer, FP16, FP32, and FP64 operations.
- Supports FMA for FP64, FP32, and FP16 (FP16 as of Vega 20).
- Warning: 0.5 ULP accurate division generates reciprocal, multiply, and fixup instructions.
- Configurable denorm and rouding modes; FP32 and FP16/FP64 configured separately.
- >= MI100: Each VALU unit also contains hardware for BF16, FP16, and FP32 GEMM.
  - MI200 adds DGEMM acceleration.

Inside a Compute Unit – Vector ALUs



	"Vega 10"	"Vega 20"	"MI100"	"MI200"
Trad. FP32 FLOP/cycle	32	32	32	32
Trad. FP64 FLOP/cycle	2	**16**	16	**32**
Packed FP16 FLOP/cycle	64	64	64	64
FP16/int16 dot Op/cycle	-	**64**	64	64
int8 dot Op/cycle	-	**128**	128	128
int4 dot Op/cycle	-	**256**	256	256
FP32 GEMM FLOP/cycle	-	-	**64**	64
FP16/int8 GEMM Op/cycle	-	-	**256**	256
BF16 GEMM FLOP/cycle	-	-	**128**	**256**
FP64 GEMM FLOP/cycle	-	-	-	**64**
Packed FP32 FLOP/cycle	-	-	-	**64**

Inside a Compute Unit – Vector Register Files

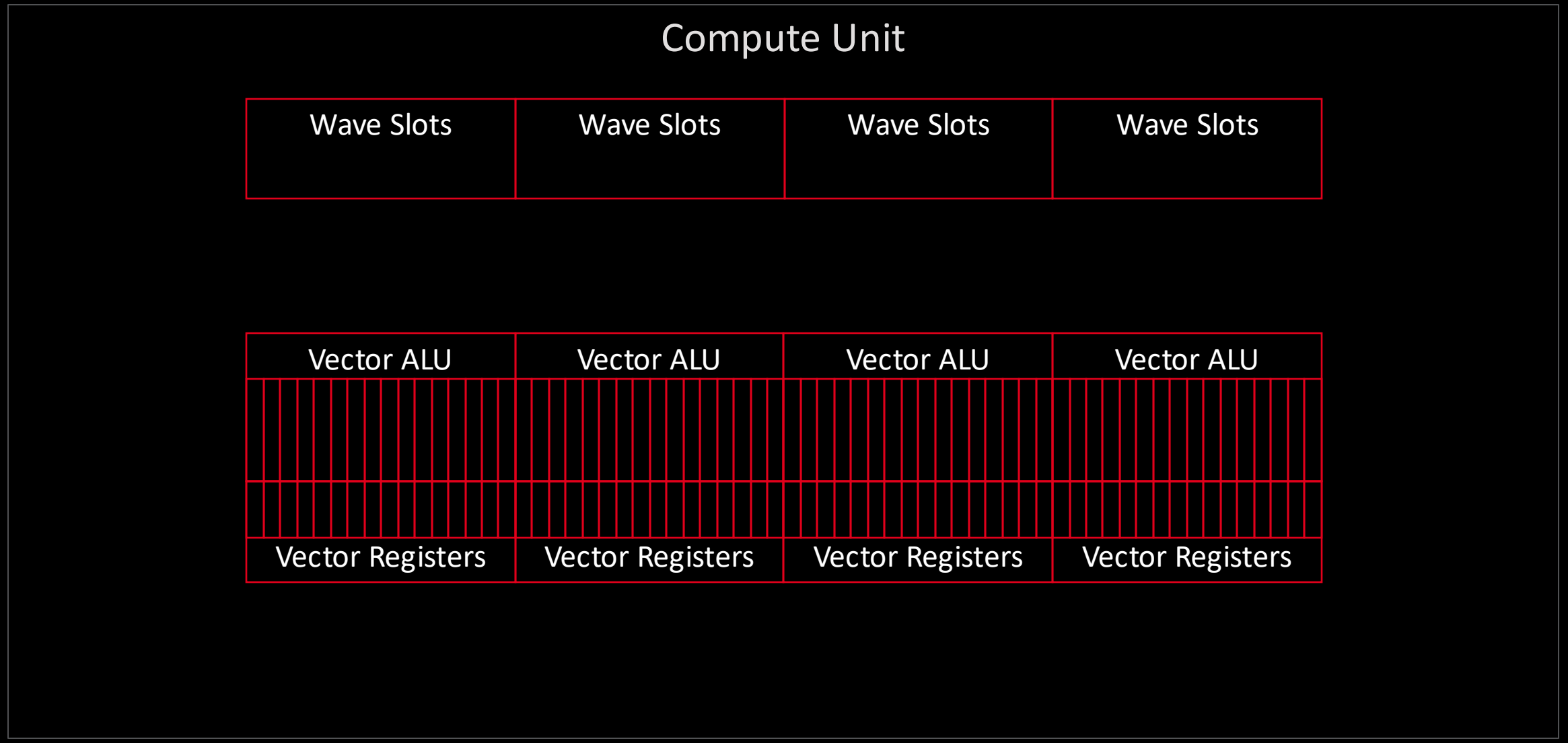


- Each VGPR is 64 registers wide. Each wavefront's VALU lane (thread) accesses one of them.
  - Data Parallel Primitive (DPP) instructions allow sourcing from another lane's register.
- All registers are 4 bytes (DWORD) wide. 64-bit values are stored in two contiguous VGPRs.
- Vega 20: 256 VGPRs per VALU.
- MI100: 256 VGPRs for traditional VALU. 256 accVGPRs for GEMM acceleration.
- MI200: Shared 512 VGPRs per VALU.
- Every wavefront can use up to 256 VGPRs (and 256 accVGPRs)

VGPRs	32	36	40	48	64	84	<= 128	> 128
Waves/SIMD-16	8	7	6	5	4	3	2	1

- Double the above register counts for MI200 if you are not using accVGPRs.

Compute Unit Internals



AMD GCN Instruction Classes

- Vector ALU (VALU): 64-wide ALU instruction.
- Vector Memory (VMEM): 64-wide memory instruction.
- Local Data Share (LDS): 64-wide memory operations to CU scratchpad.
- Scalar ALU (SALU): ALU operation if every thread is working on identical data.
- Scalar Memory (SMEM): Identical memory operation in all lanes.
- Branch: Used if the entire wavefront wants to branch.
- Internal: Wavefront bookkeeping. NOPs, workgroup barriers, wait-for-memory, etc.

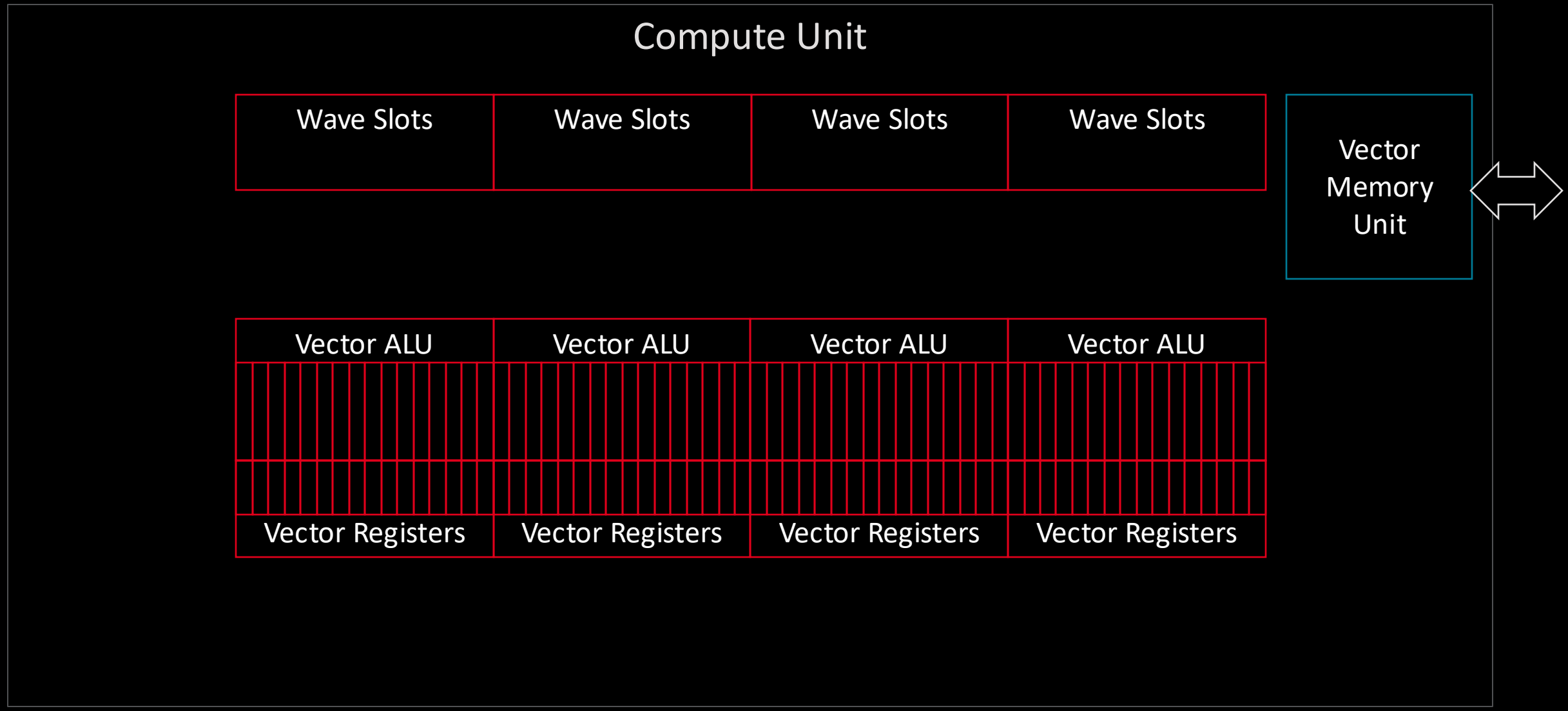
Inside a Compute Unit – Vector Memory

Vector Memory Unit

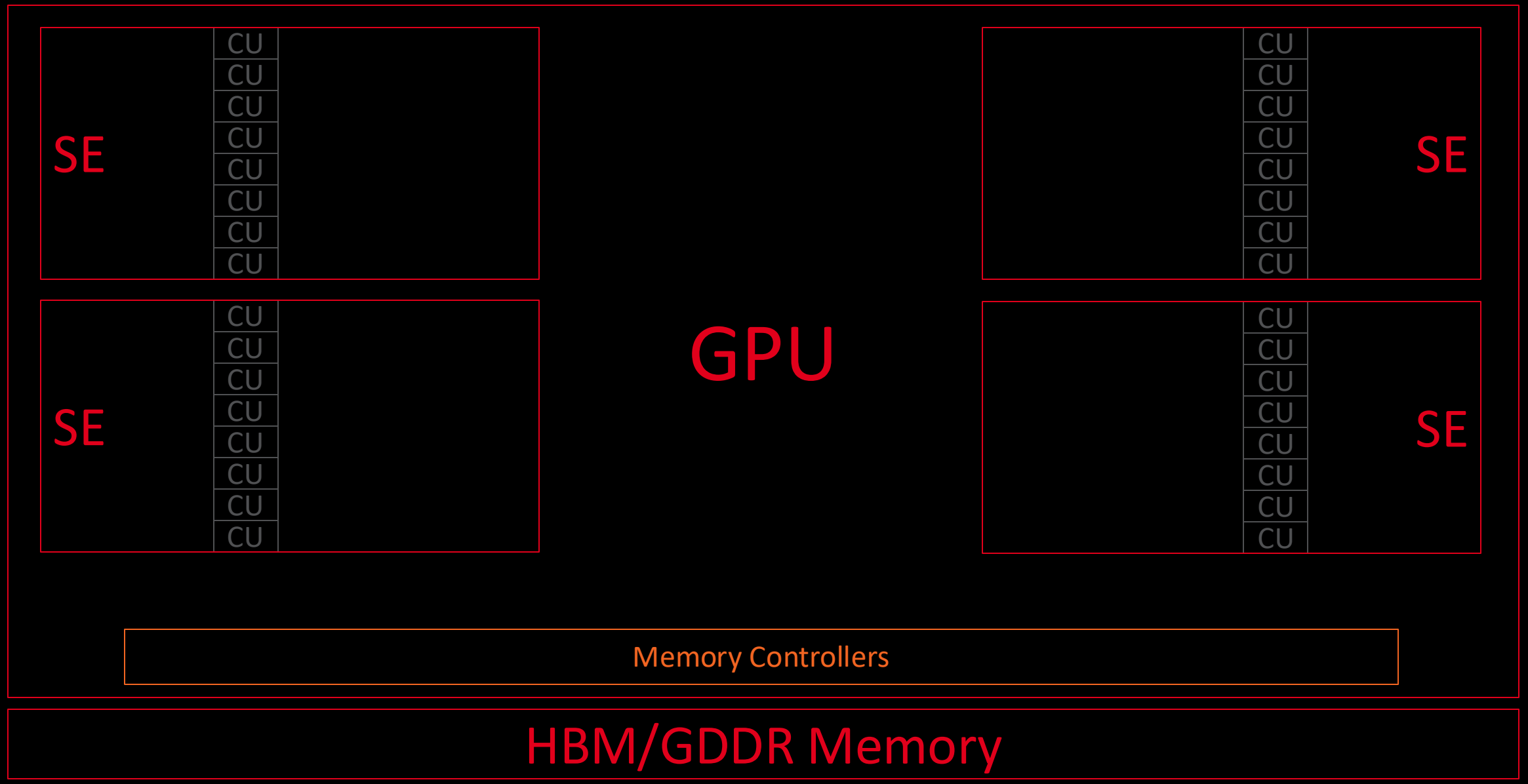
- Vector memory operations from all 4 wave slot groups are routed to this unit.
  - Transfers data between global memory and VGPRs
- A wavefront issuing a VMEM instruction requires the VMEM unit for 4 cycles to issue addresses.
- Can handle uncoalesced addresses (e.g., each lane to a different address).
- Includes request coalescing logic to increase performance / minimize requests out of the CU.
- Can write out 64B per cycle, or read in 64B per cycle.
- Connects to a 16 KiB vector L1 data cache, and out to further memory system.

- Includes logic for Buffer memory accesses, which can perform complex offsetting calculations automatically.

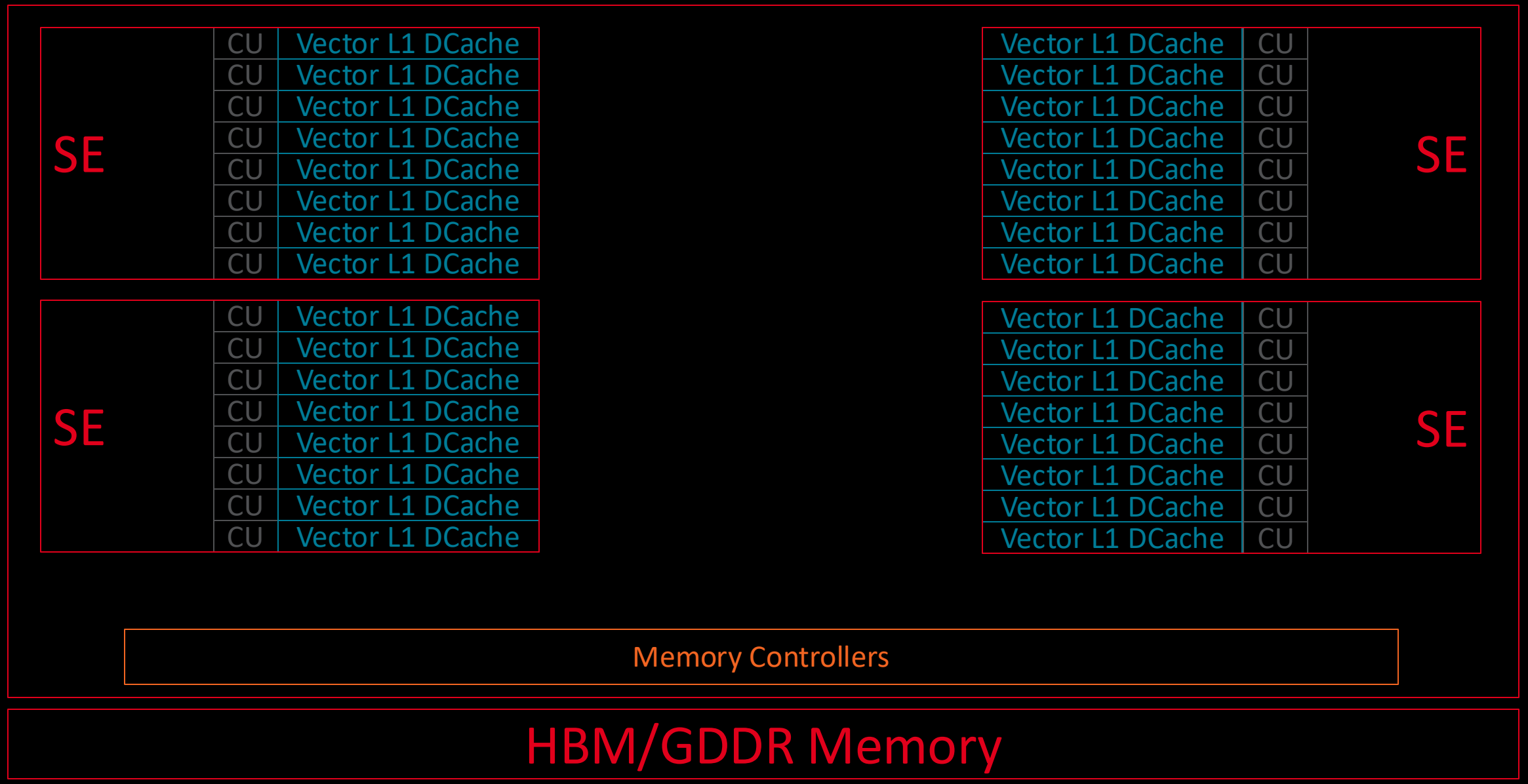
Compute Unit Internals



On-chip Memory System



On-chip Memory System



Per-CU Vector L1 Data Cache (TCP)

Read 64B/cycle
Write 64B/cycle
(or send read addr)

Vector L1 Data Cache

- 16 KiB Read/Write Data Cache
- 64B cache lines, 64-way set associative
  - 64 MSHR in each set
- Write-through, Write-allocate if whole line is dirty
  - If SLC & GLC bits are set in VMEM instruction, write-no-allocate
- Performs write-combining of multiple stores from a single wave
- VMEM GLC bit allows bypass of vL1D
- VMEM instructions controls replacement policy:
  - Default to LRU replacement
  - VMEM SLC bit set causes misses to insert as LRU
- Non-inclusive of L2 cache
- Virtually indexed, physically tagged

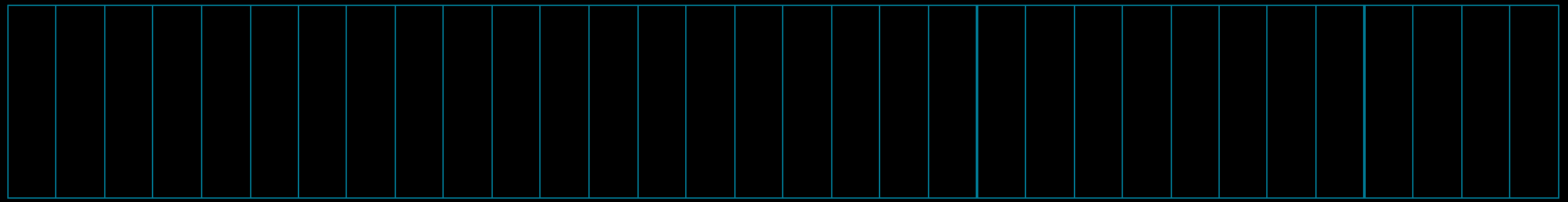
Vector L1 DTLB

- 32-entry, fully associative, LRU replacement
- Supports 4KiB, 2MiB, and other page table entry sizes
- Up to 16 VMIDs in use at once
- 2 translations/cycle
- 64 outstanding misses

AMD GCN Instruction Classes

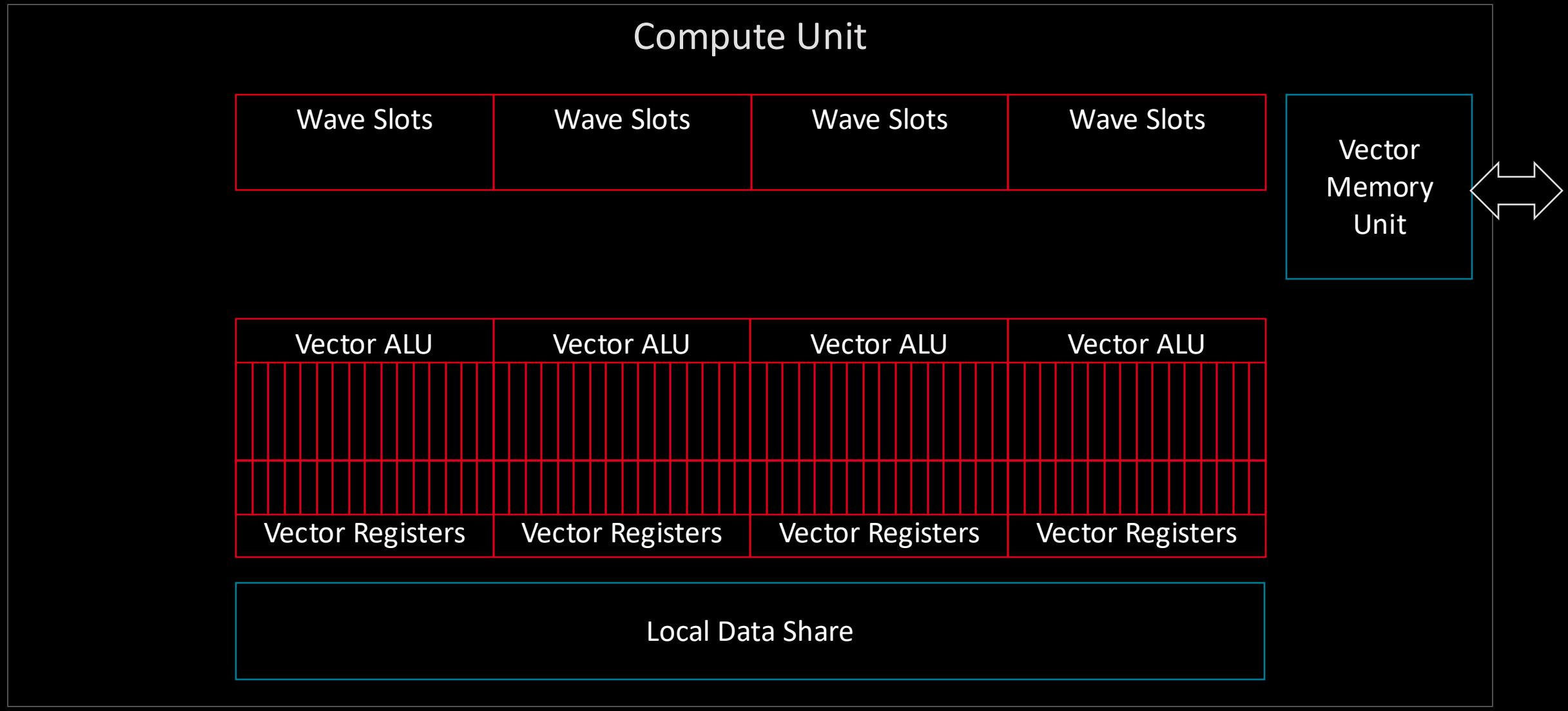
- Vector ALU (VALU): 64-wide ALU instruction.
- Vector Memory (VMEM): 64-wide memory instruction.
- Local Data Share (LDS): 64-wide memory operations to CU scratchpad.
- Scalar ALU (SALU): ALU operation if every thread is working on identical data.
- Scalar Memory (SMEM): Identical memory operation in all lanes.
- Branch: Used if the entire wavefront wants to branch.
- Internal: Wavefront bookkeeping. NOPs, workgroup barriers, wait-for-memory, etc.

Inside a Compute Unit – Local Data Share



- 64 KiB scrachpad memory accessible from any SIMD-16 in this CU.
- 32 banks: useful for swizzling data between vector lanes or uncoalesced accesses.
  - 128B per cycle results in a full wavefront serviced every 2 cycles.
- Includes atomic units for logical, integer, FP32. MI200 adds FP64.
- LDS network used for inter-lane swizzle and permute operations without needing storage space.
- All wavefronts within a workgroup can access the same range of LDS.
  - Requested LDS storage per workgroup can thus limit CU occupancy

Compute Unit Internals



AMD GCN Instruction Classes

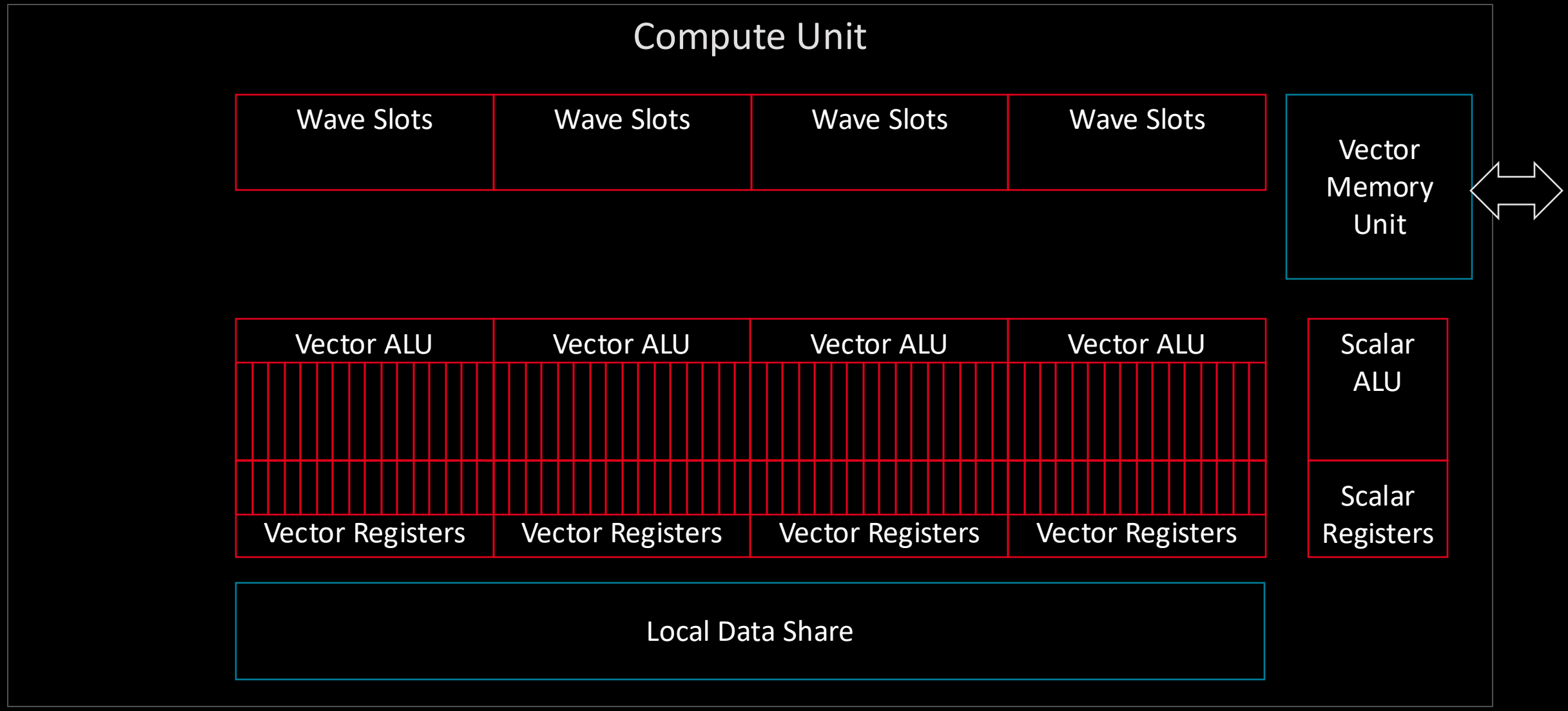
- Vector ALU (VALU): 64-wide ALU instruction.
- Vector Memory (VMEM): 64-wide memory instruction.
- Local Data Share (LDS): 64-wide memory operations to CU scratchpad.
- Scalar ALU (SALU): ALU operation if every thread is working on identical data.
- Scalar Memory (SMEM): Identical memory operation in all lanes.
- Branch: Used if the entire wavefront wants to branch.
- Internal: Wavefront bookkeeping. NOPs, workgroup barriers, wait-for-memory, etc.

Inside a Compute Unit – Scalar ALU

- Integer arithmetic and logical operations used when all lanes of a wavefront doing the same thing.
- When using SALU instruction, only one operation is performed (no duplication of computation).
- Especially useful for setting up wavefront-wide control flow.
  - Converged branches, function calls, etc.

- 800 SGPRs available for wavefronts in each wave slot group.
  - SGPRs can also be read by VALU instructions; all lanes see the same value.

Compute Unit Internals



AMD GCN Instruction Classes

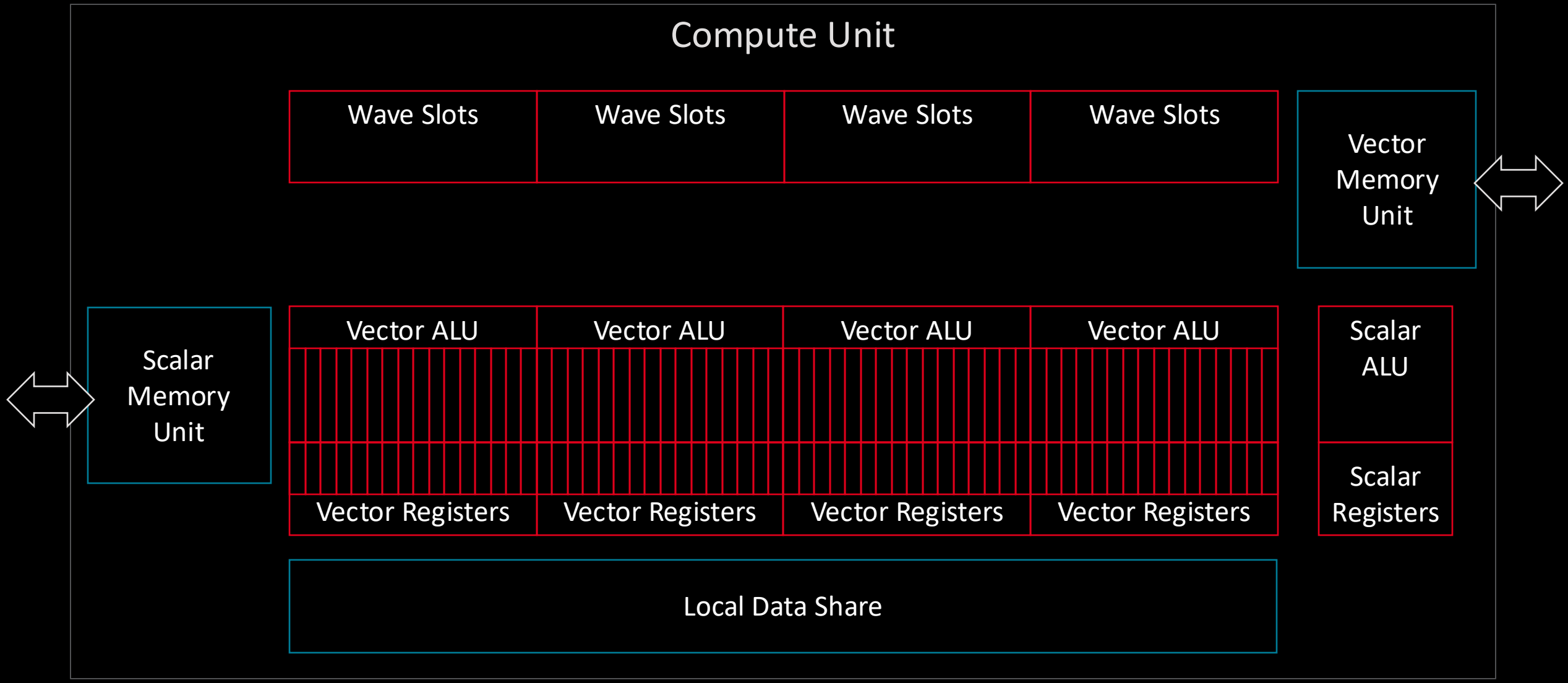
- Vector ALU (VALU): 64-wide ALU instruction.
- Vector Memory (VMEM): 64-wide memory instruction.
- Local Data Share (LDS): 64-wide memory operations to CU scratchpad.
- Scalar ALU (SALU): ALU operation if every thread is working on identical data.
- Scalar Memory (SMEM): Identical memory operation in all lanes.
- Branch: Used if the entire wavefront wants to branch.
- Internal: Wavefront bookkeeping. NOPs, workgroup barriers, wait-for-memory, etc.

Inside a Compute Unit – Scalar Memory

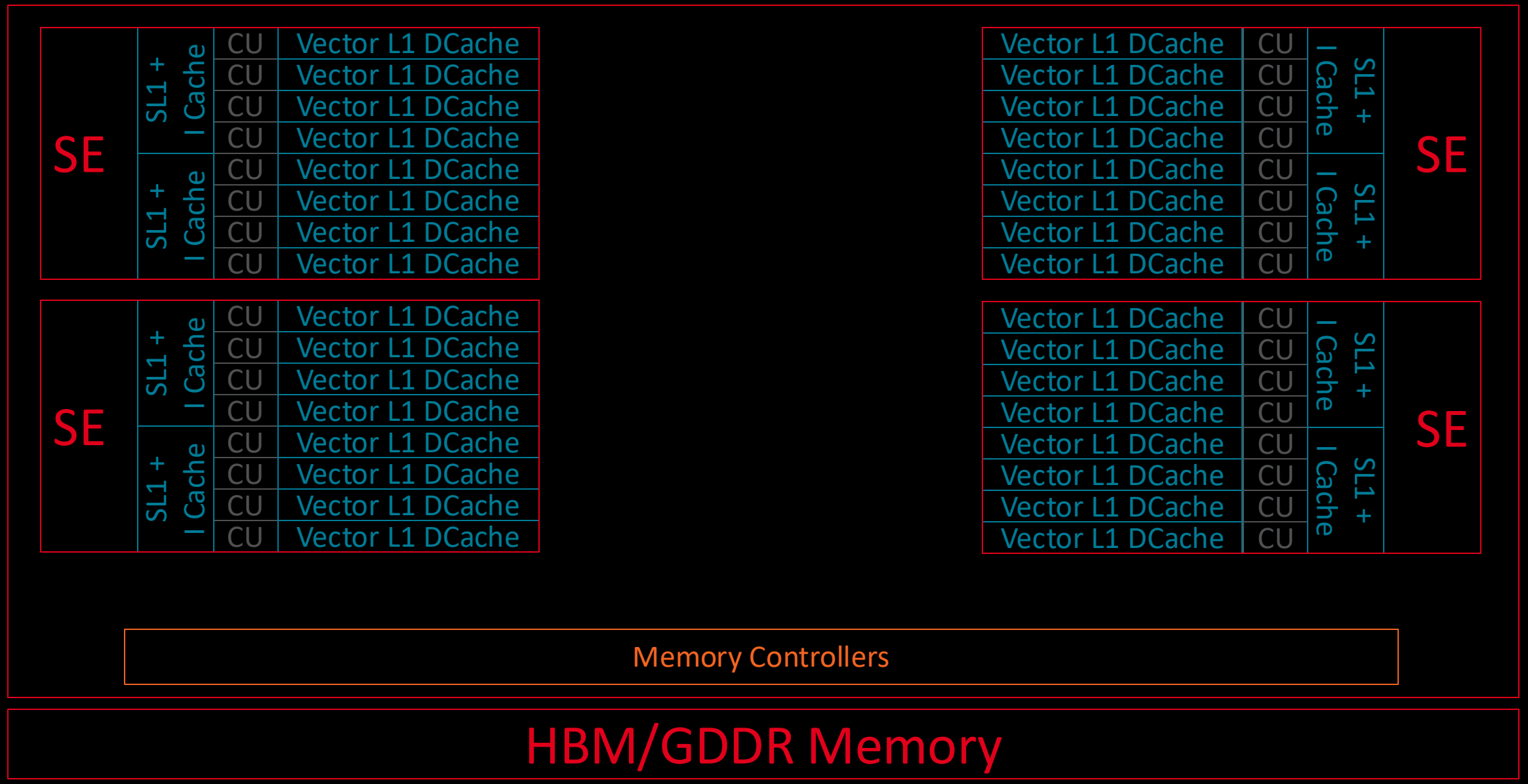
Scalar Memory Unit

- Scalar memory operations from all 4 wave slot groups are routed to this unit.
  - Transfers data between global memory and SGPRs.
- Supports reads, writes, and logical/integer atomics.
- Used if you want to do a wavefront-wide access, rather than each lane doing individual accesses.
- Can read or write 16B/cycle to memory system.
- Connects to a 16 KiB scalar L1 data cache, and out to further memory system.
  - Different cache than the vL1D. But shared with three other CUs.

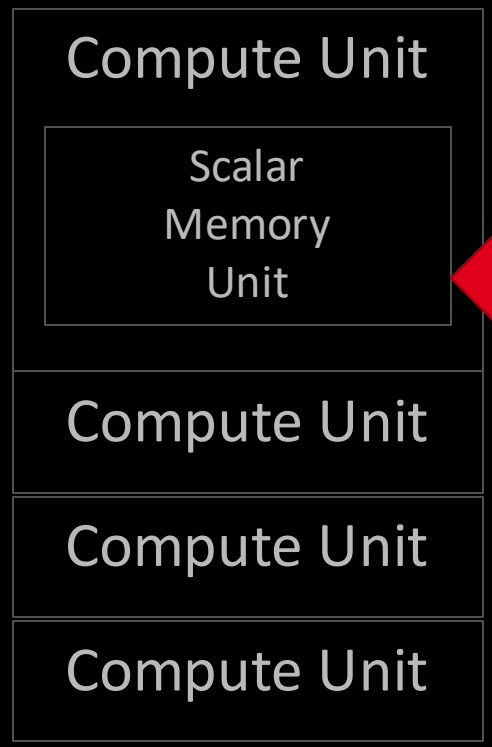
Compute Unit Internals



On-chip Memory System



One Scalar L1 Data Cache shared by 4 CUs



Per CU
Read 16B/cycle
Write 16B/cycle
(or send read addr)

Scalar L1 Data Cache

- 16 KiB Read/Write Data Cache
- 64B cache lines, 4-way set associative, 4 banks
  - 32 MSHR per bank
- Default: Write-back, write-no-allocate
  - If GLC bit is set in SMEM write, write-through + invalidate
  - If GLC bit is set in SMEM read, miss to L2 + invalidate
- LRU replacement when not forcing miss
- Non-inclusive of L2 cache
- Virtually indexed, physically tagged

Scalar L1 DTLB

- 32-entry, fully associative, LRU replacement
- Supports 4KiB, 2MiB, and other page table entry sizes
- Up to 16 VMIDs in use at once
- 2 translations/cycle
- 64 outstanding misses

AMD GCN Instruction Classes

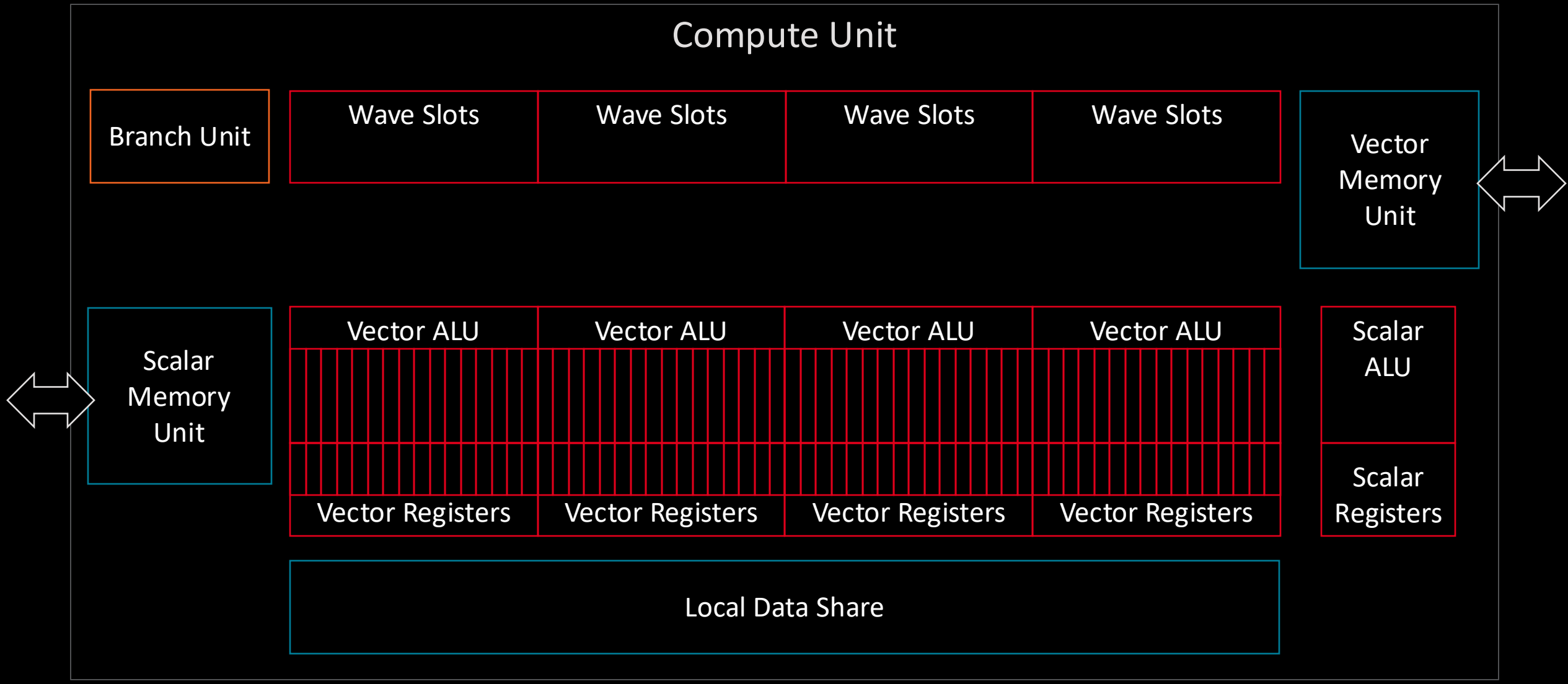
- Vector ALU (VALU): 64-wide ALU instruction.
- Vector Memory (VMEM): 64-wide memory instruction.
- Local Data Share (LDS): 64-wide memory operations to CU scratchpad.
- Scalar ALU (SALU): ALU operation if every thread is working on identical data.
- Scalar Memory (SMEM): Identical memory operation in all lanes.
- Branch: Used if the entire wavefront wants to branch.
- Internal: Wavefront bookkeeping. NOPs, workgroup barriers, wait-for-memory, etc.

Inside a Compute Unit – Branch Unit

Branch Unit

- Unit used specifically to handle wavefront-wide branching.
- Per-lane “branching” is handled by changes to the EXEC mask, a 64-bit predicate register.
  - If your lane’s bit is 0 in the EXEC mask, your VALU and VMEM ops are NOPs.
- Arbitrary changes in program counter (e.g. jumps, function calls) handled with branch instructions.
- Similarly, if software sees that all lanes are disabled, it can choose to branch around a region of code.
  - This is **not** handled automatically by the hardware. Requires branch instruction
- Unit also used to send messages / interrupts to the host.

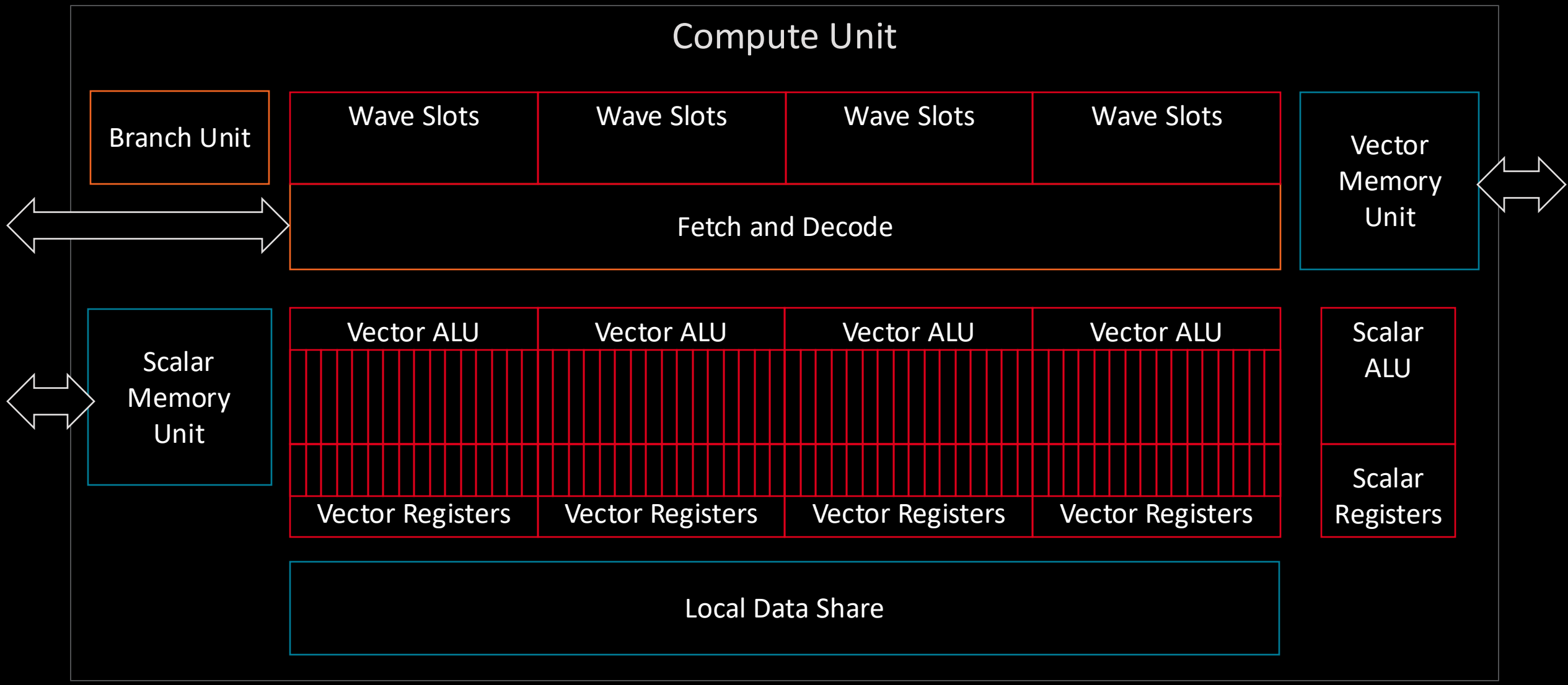
Compute Unit Internals



AMD GCN Instruction Classes

- Vector ALU (VALU): 64-wide ALU instruction.
- Vector Memory (VMEM): 64-wide memory instruction.
- Local Data Share (LDS): 64-wide memory operations to CU scratchpad.
- Scalar ALU (SALU): ALU operation if every thread is working on identical data.
- Scalar Memory (SMEM): Identical memory operation in all lanes.
- Branch: Used if the entire wavefront wants to branch.
- Internal: Wavefront bookkeeping. NOPs, workgroup barriers, wait-for-memory, etc.

Compute Unit Internals



One L1 Instruction Cache shared by 4 CUs

Per CU
Read 32B/cycle

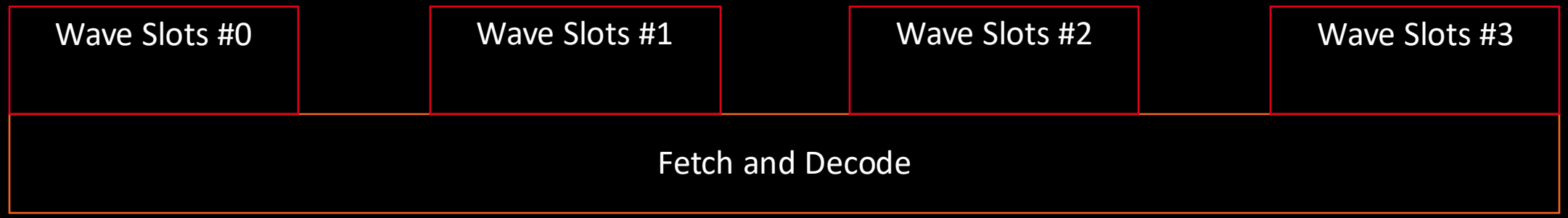
L1 Instruction Cache

- 32 KiB Read-only Instruction Cache
- 64B cache lines, 4-way set associative, 4 banks
  - 64 MSHR per bank
- LRU replacement
- Non-inclusive of L2 cache
- Virtually indexed, virtually tagged, entries include VMID
  - Only accesses ITLB on cache miss

L1 ITLB

- 32-entry, fully associative, LRU replacement
- Supports 4KiB, 2MiB, and other page table entry sizes
- Up to 16 VMIDs in use at once
- 2 translations/cycle
- 64 outstanding misses

Inside a Compute Unit – Fetch and Decode Unit



- Fetch logic only looks at one set of wave slots per cycle. Round robins through them.
  - e.g. Looks for wavefronts to run in Wave Slots #0 in cycle 0, Wave Slots #1 in cycle 1, etc.
- Executes all waves in-order, and only executes (at most) 1 instruction/wave at each scheduling interval
- Every cycle, can issue one instruction to **each** other unit in the CU:
  - VALU
  - VMEM
  - SALU/SMEM
  - LDS
  - Branch

Example of Issuing Multiple Instructions per Cycle

- Our example code from earlier:

- If we had 5 wavefronts in this wave slot group, each ready to run a different one of the highlighted instructions, we could issue 5 instructions this cycle.

```
s_load_dwordx4 s[0:3], s[4:5], 0
v_lshlrev_b64 v[2:3], 2, v[2:3]
s_waitcnt lgkmcnt(0)
s_add_u32 s0, 16, s0
s_addc_u32 s1, 0, s1
v_add_co_u32 v0, s0, v2
v_addc_co_u32 v1, s1, v3
global_load_dword v4, v[0:1]
v_add_co_u32 v0, s2, v2
v_addc_co_u32 v1, s3, v3
s_waitcnt vmcnt(0)
global_store_dword v[0:1], v4
s_endpgm
```

Scalar Memory

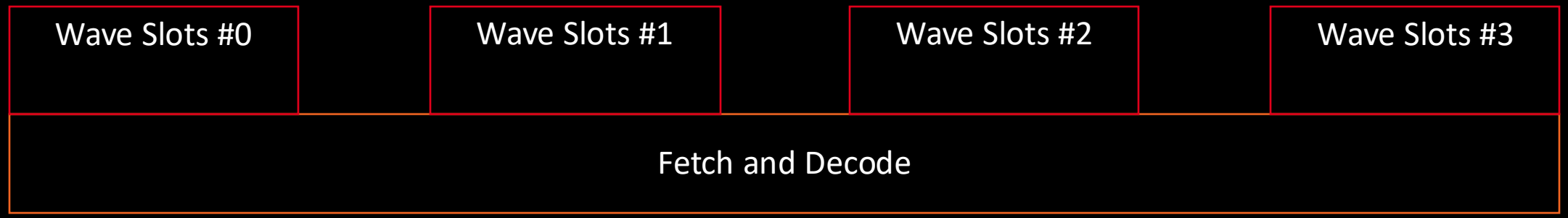
Vector ALU

Scalar ALU

Vector Memory

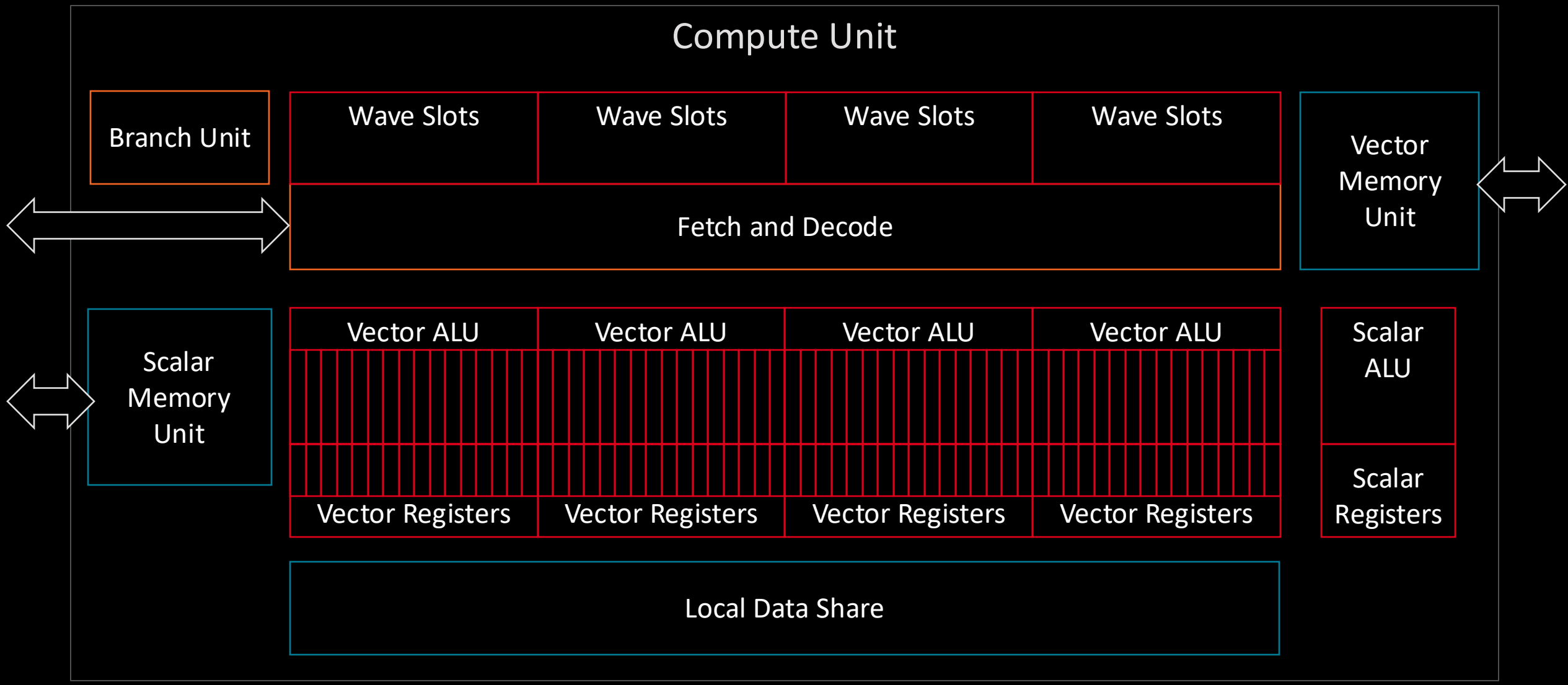
Internal

Inside a Compute Unit – Fetch and Decode Unit



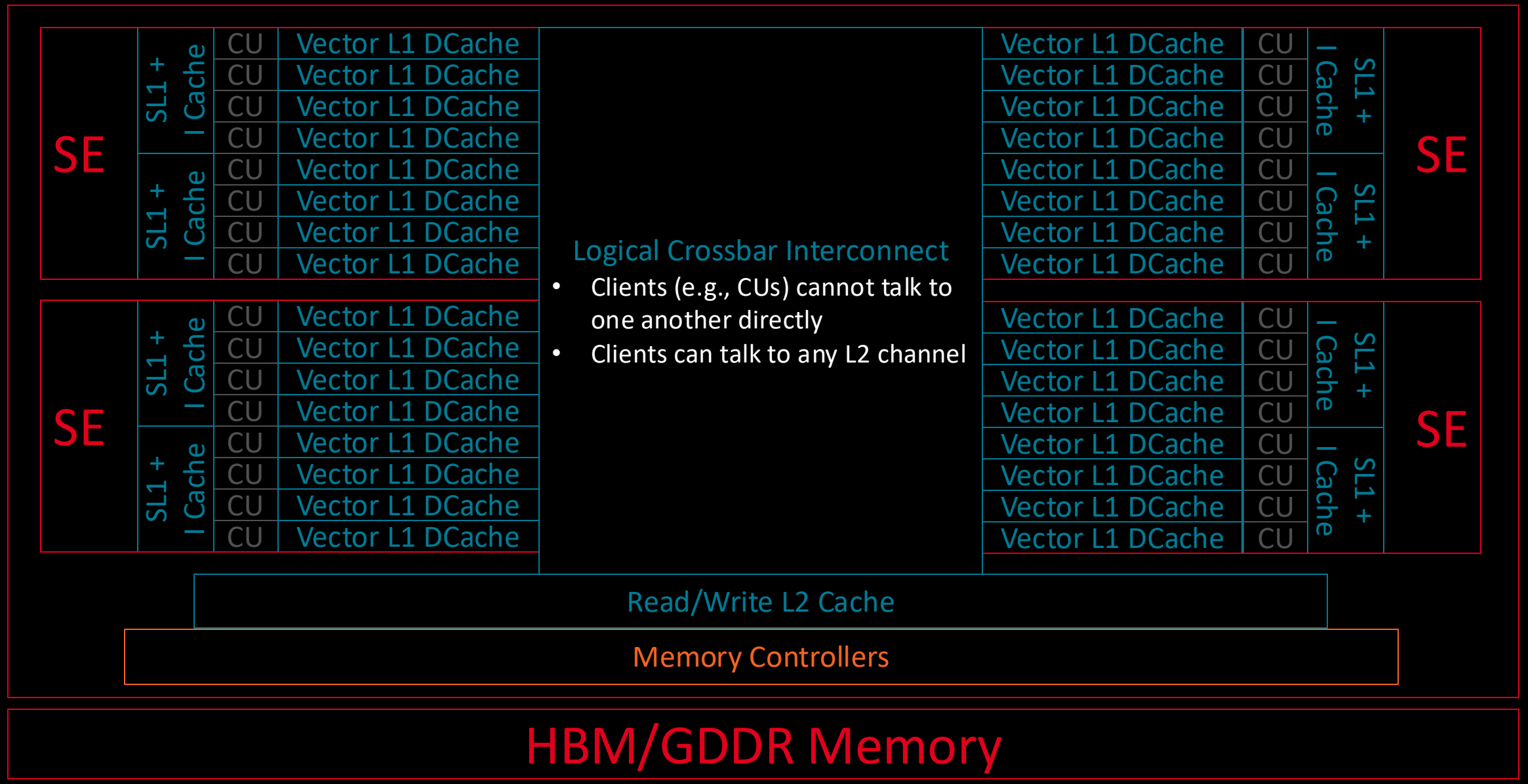
- If there are multiple ready wavefronts vying for the same resource, fetch priority:
  - Normally: oldest ready wave first.
  - Can be overridden by S_SETPRIO instruction: set your wavefront's priority higher or lower.
  - Hardware also overrides this logic for "soft clauses" of memory operations.
    - HW tries to issue consecutive memory operations back-to-back to increase memory system efficiency.
    - Alternate way to read this: try to "bunch up" your memory operations in your applications.
- "Internal" operations:
  - Wavefront stall, priority operations like S_NOP, S_SLEEP, S_SETPRIO
  - Waiting for memory accesses to complete: S_WAITCNT
  - Waiting at workgroup-wide barrier: S_BARRIER
- 16 hardware workgroup-wide barriers in each CU; limits us to 16 >1-wave workgroups.

Compute Unit Internals



On-chip Memory System

On-chip Memory System



Shared L2 Cache is a Coherence Point for Graphics Clients

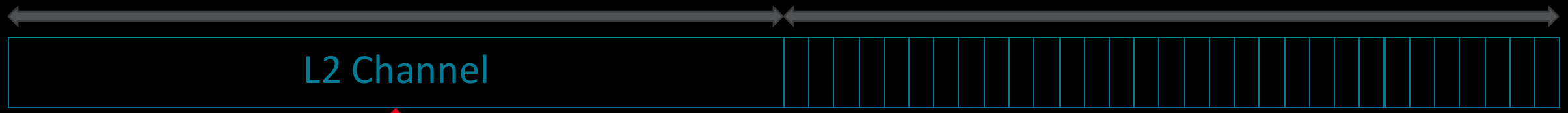
L2 Cache (TCC)

- 16-way set associative
- 64B lines (128B in MI-200)
- Write-back, write allocate
- Default: LRU replacement policy
  - Possible to insert misses at LRU (e.g. SLC bit in VMEM ops)
- Physically indexed, physically tagged
- Supports atomic operations:
  - Vega 10, Vega 20: Integer and logical
  - MI100: FP32 and FP16 add
  - MI200: FP64 add

	Vega 10	Vega 20	MI100	MI200
L2 Cache Size	4 MiB	4 MiB	8 MiB	8 MiB
Per-channel L2 Bandwidth	64B/cycle read 64B/cycle write	64B/cycle read 64B/cycle write	64B/cycle read 64B/cycle write	128B/cycle read 64B/cycle write

256B Channel Interleaving

Vega 10, Vega 20: 16 channels
MI100, MI200: 32 channels



Normally: 1 EA per L2 Channel
Vega 20: 2 EA per channel

32 B/cycle each direction
MI-200: 64 B/cycle

Shared L2 TLB for Graphics Core Clients

Description for “GPUVM” page tables; IOMMU-based virtualization uses further ATC translations

L2 TLB

4 KiB Pages

- 4096 entries (16,384 in MI200)
- 2-way set associative, 4 banks (8 in MI200)
- LRU replacement
- 4 translations / cycle (8 in MI200)

2 MiB Pages

- 32,768 entries
- 2-way set associative, 4 banks (8 in MI200)
- LRU replacement
- 4 translations / cycle (8 in MI200)

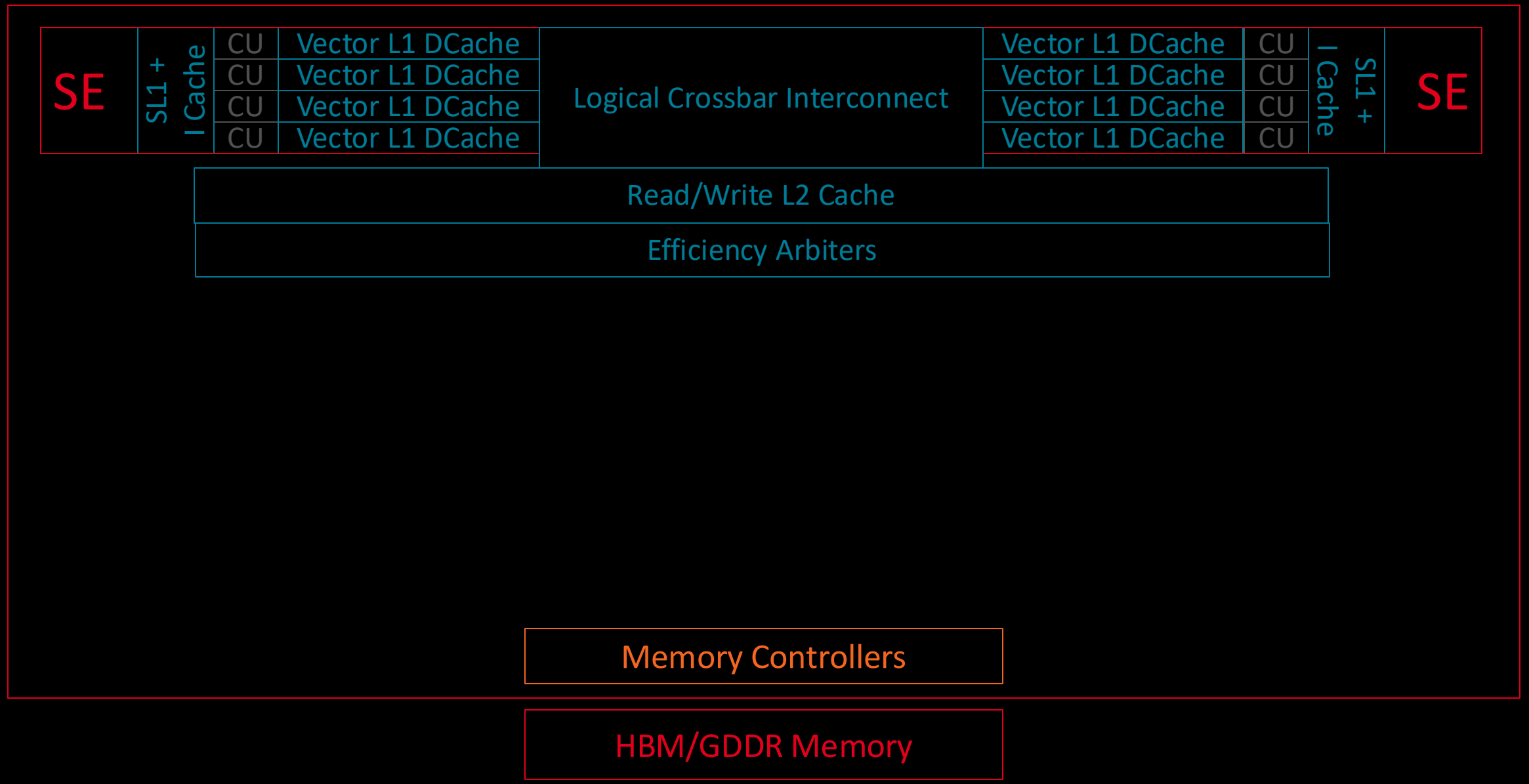
Page Table Walker

- 2048-, 1024-, and 512-entry caches for page directory entries
- Parallel lookups into each cache, 1 per cache per cycle
- One request to memory per clock, up to 128 in flight

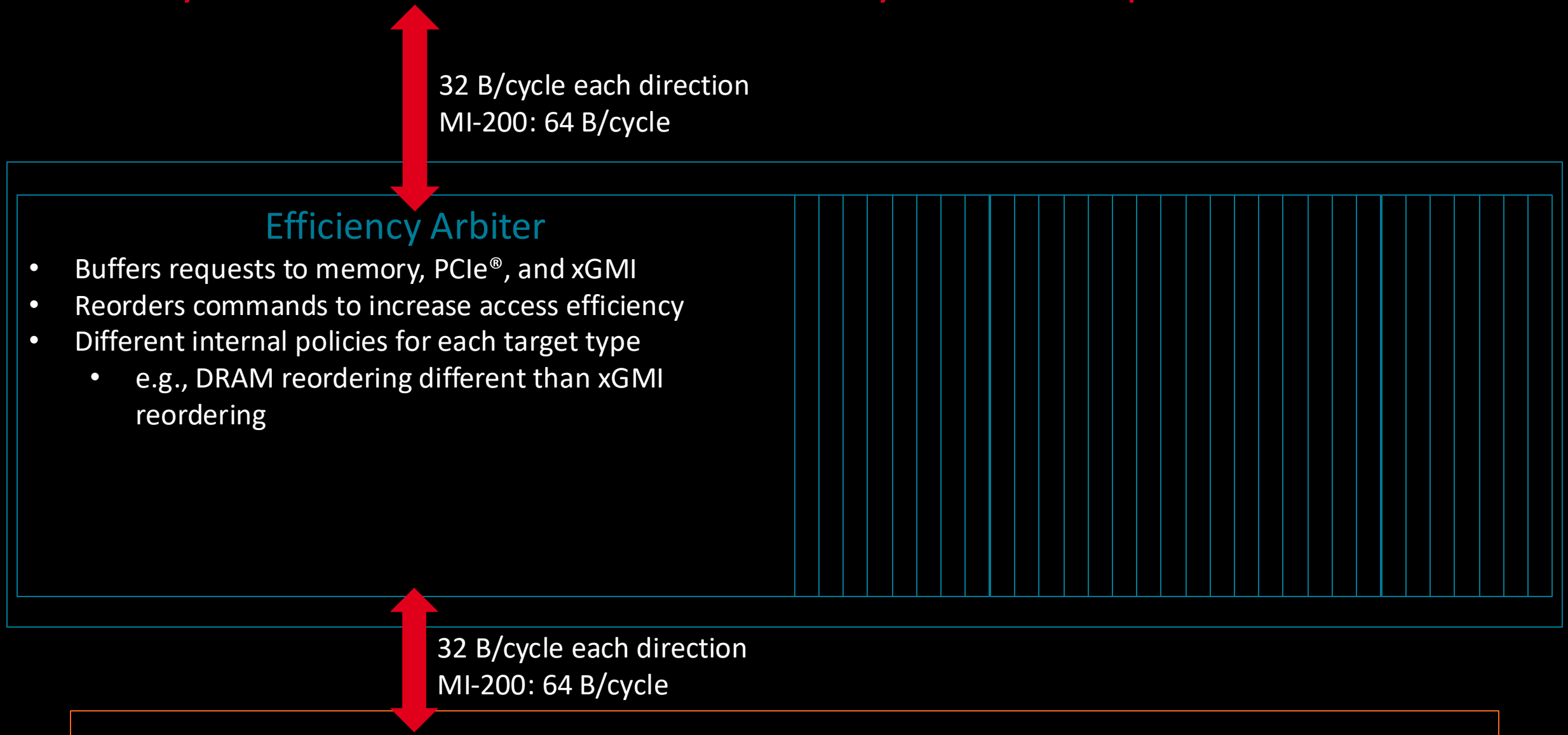
L2 Cache

GPU Memory System – Getting Off Chip

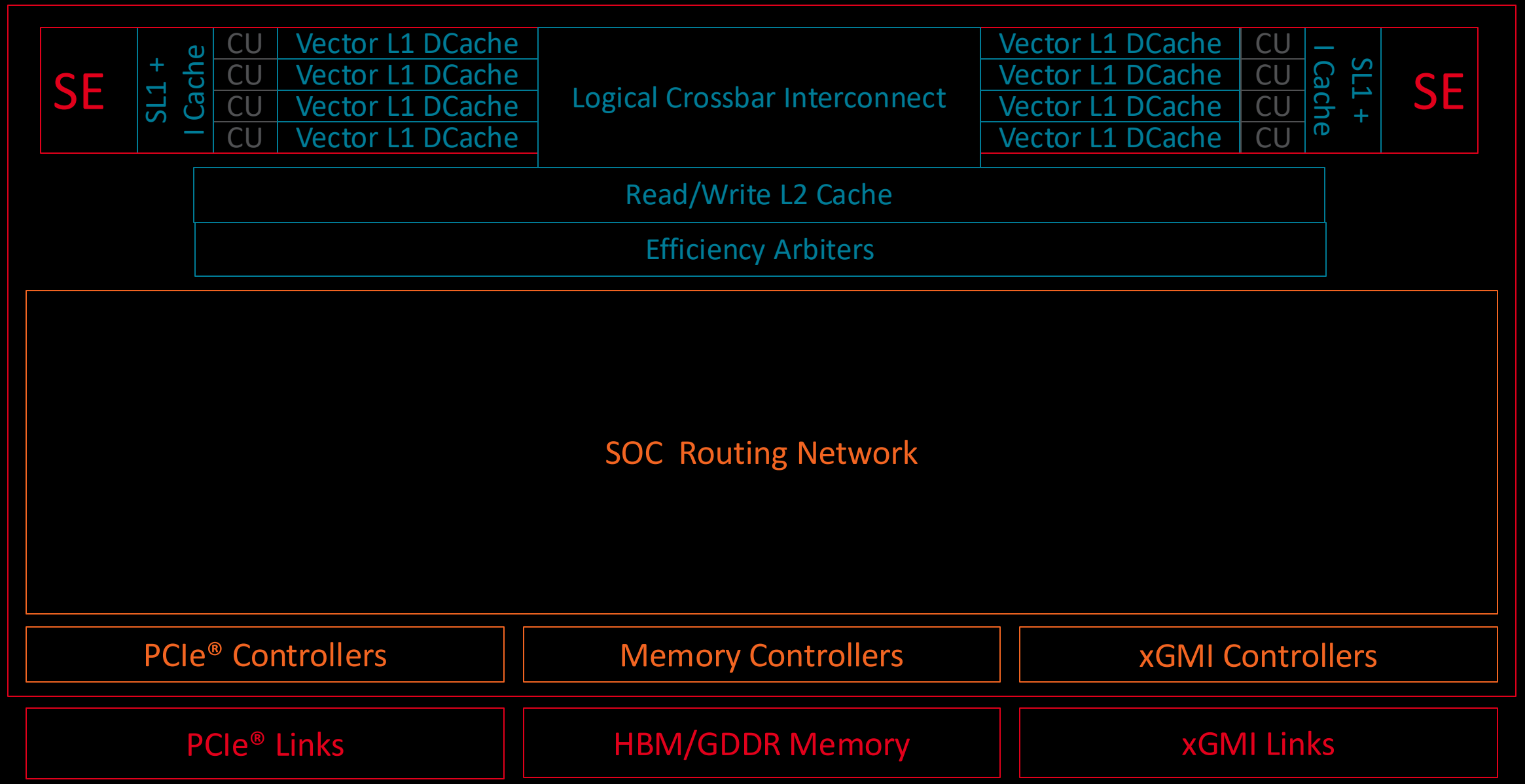
On-chip Memory System



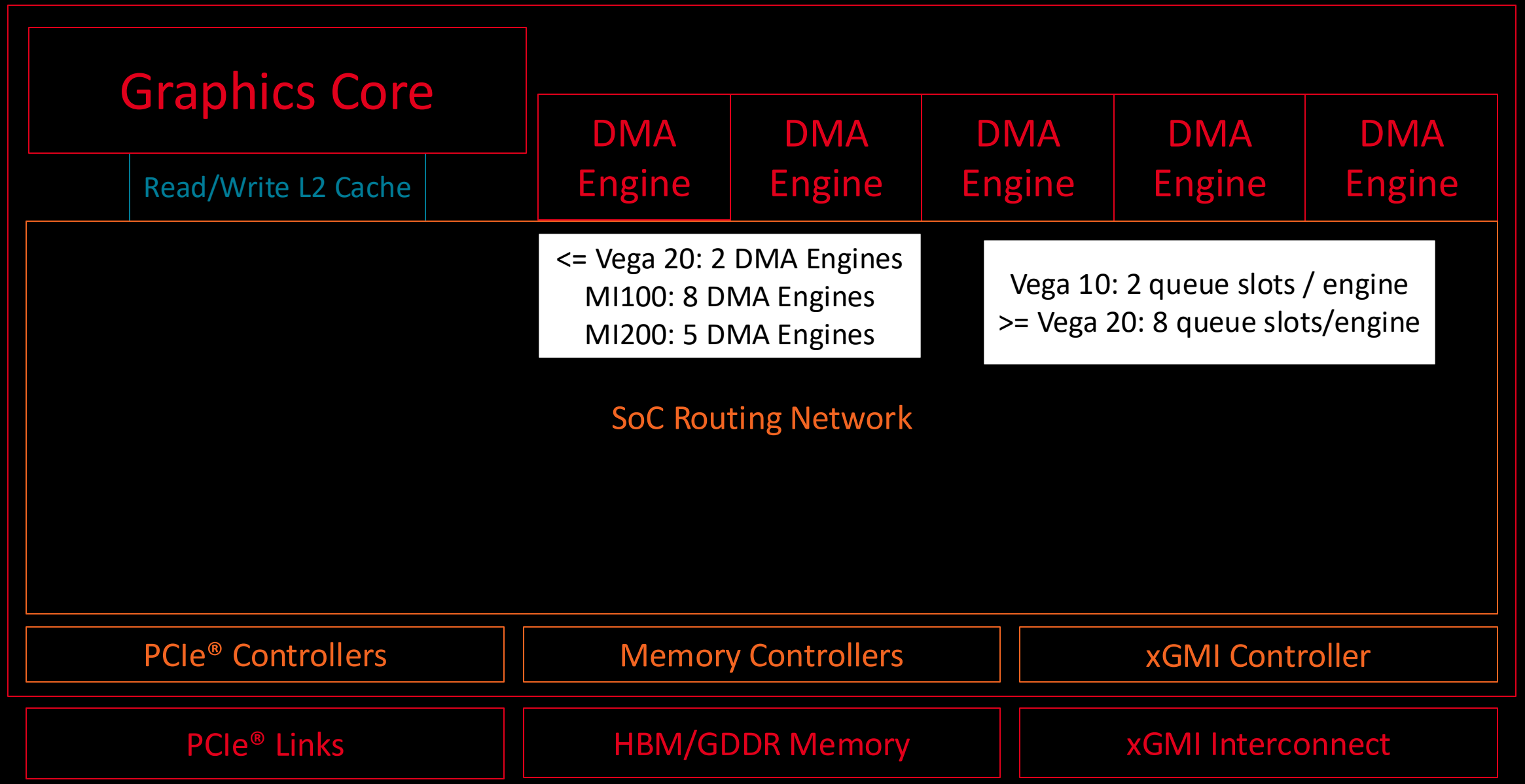
Efficiency Arbiters Order Accesses to Memory and Off-Chip Buses



On-chip Memory System



On-chip Memory System



Comparing AMD GPUs

AMD “Graphics Core Next” GPU Generations

- Multiple “GCN” generations released since then
  - 2012: “Southern Islands”
  - 2013: “Sea Islands”
  - 2015: “Volcanic Islands”
  - 2017: “Vega”

- Some things stay the same across these generations:
  - Compute Unit (CU) parallelism
    - 64-wide hardware vector processors
  - Mostly in-order execution
    - Fine-grained multithreading for latency hiding

- Some things change:
  - Opcodes added & removed
    - Changes to how GPU interacts with compilers & drivers
  - FP64 execution rate, packed FP16 math, GEMM & dot product acceleration, inter-lane communication mechanisms

GCN Generations – Compiler and Driver View

The AMDGPU LLVM compiler backend defines GCN generations numerically, rather than with code names.

Major ISA versions are covered in new ISA manuals.

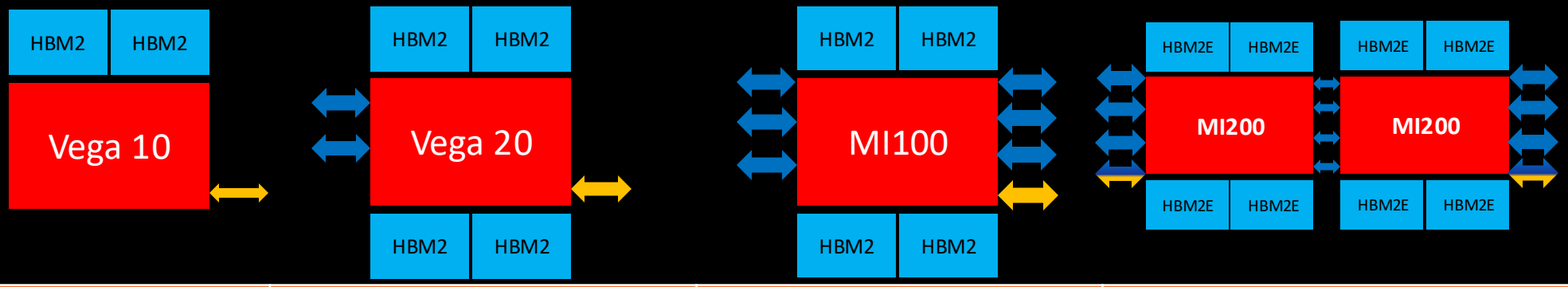
Different minor ISA versions tell the compiler how to work around bugs, how to generate more optimal code for a device class, how to enable specific features, etc.

For example:

- 801 vs. 802: Former allows precise page faults
- 802 vs. 803: Former uses SW workarounds for HW bugs
- 900→906: New instructions for dot products
- 906→908: SGEMM and HGEMM Acceleration

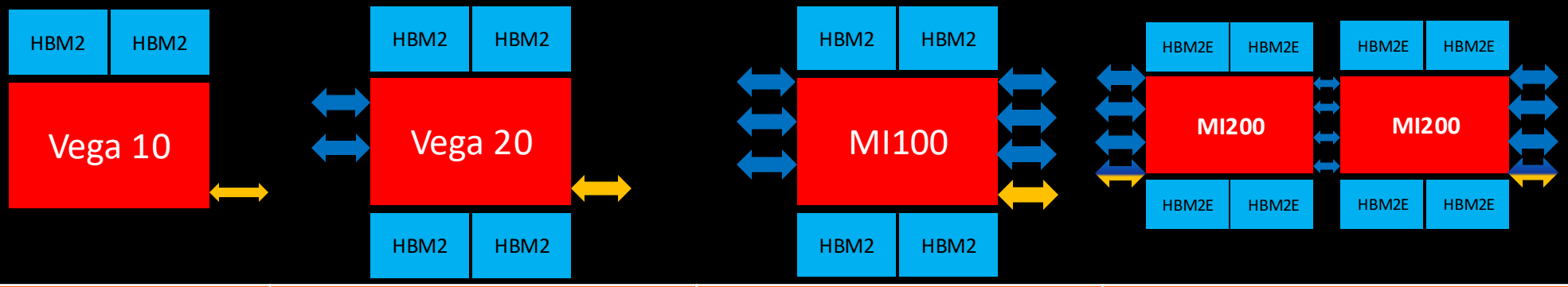
- gfx6 (“Southern Islands”)
  - gfx600 – “Tahiti” dGPU
  - gfx601 – “Pitcairn” dGPU
- gfx7 (“Sea Islands”)
  - gfx701 – “Hawaii” dGPU
- gfx8 (“Volcanic Islands” / GCN3 ISA)
  - gfx801 – “Carrizo” iGPU
  - gfx802 – “Tonga” dGPU
  - gfx803 – “Fiji” and “Polaris” dGPUs
- gfx9 (“Vega” ISA)
  - gfx900 – “Vega 10” dGPU
  - gfx902 – “Raven Ridge” iGPU
  - gfx904 – “Vega 12” dGPU
  - gfx906 – “Vega 20” dGPU
  - gfx908 – “MI100” dGPU
  - gfx90a – “MI200” dGPU

AMD GPU Speeds Comparison



	"Vega 10"	"Vega 20"	"MI100"	"MI200"
	gfx900	gfx906	gfx908	gfx90a
Compute Units	64	64	128 (120)	112 (110 or 96) per die
Peak Frequency (GHz)	1.5 @ 220W TGP	1.8 @ 225 W TGP	(Estimate) 1.55 @ 255 W TGP	(Estimate) 1.3 @ 215 W TGP
Trad. FP32 FLOP/CU/cycle	128	128	128	128
Trad. FP64 FLOP/CU/cycle	8	**64**	64	**128**
Packed FP16 FLOP/CU/cycle	256	256	256	256
FP16/int16 dot Op/CU/cycle	-	**256**	256	256
int8 dot Op/CU/cycle	-	**512**	512	512
int4 dot Op/CU/cycle	-	**1024**	1024	1024
FP32 GEMM FLOP/CU/cycle	-	-	**256**	256
FP16/int8 GEMM Op/CU/cycle	-	-	**1024**	1024
BF16 GEMM FLOP/CU/cycle	-	-	**512**	**1024**
FP64 GEMM FLOP/CU/cycle	-	-	-	**256**
Packed FP32 FLOP/CU/cycle	-	-	-	**256**

AMD GPU Feeds Comparison



	"Vega 10"	"Vega 20"	"MI100"	"MI200"
Peak HBM DRAM BW	484 GB/s	**1 TB/s**	**1.2 TB/s (Est.)**	**1.6 TB/s per die (Est.)**
Max HBM Memory Size	16 GiB	**32 GiB**	32 GiB	**64 GiB** per die
Bidirectional PCIe® Speed	30 GB/s	**60 GB/s (100 with ESM)**	60 GB/s (100 with ESM)	60 GB/s (100 with ESM)
Infinity Fabric™ Links	-	**2**	**6**	**8 (out of module)**
Bidirectional xGMI Speed	-	**100 GB/s**	100 GB/s	**128 GB/s**
Virtual / Physical Address Bits	48 / 40	48 / **44**	48 / 44	48 / **48**
Vector Register Space / CU	256 KiB	256 KiB	**256 KiB + 256 KiB GEMM Regs**	**512 KiB**
L2 Cache Size	4 MiB	4 MiB	**8 MiB**	8 MiB per die
L2 Cache Line Size	64 bytes	64 bytes	64 bytes	**128 bytes**
L2 Bandwidth	1 KiB/cycle read 1 KiB/cycle write	1 KiB/cycle read 1 KiB/cycle write	**2 KiB/cycle read** **2 KiB/cycle write**	**4 KiB/cycle read** 2 KiB/cycle write
L2 Atomic Math Ops	Integer	Integer	**+FP32, FP16**	**+FP64**
L2 TLB Reach (2 MiB pages)	64 GiB	64 GiB	64 GiB	64 GiB
L2 TLB Translation BW	4 lookups / cycle	4 lookups / cycle	4 lookups / cycle	**8 lookups / cycle**

Software Terminology and Translation Key

Nvidia/CUDA Terminology	AMD Terminology	Description
Kernel	Kernel	Functions launched to the GPU that are executed by multiple parallel workers on the GPU. Kernels can work in parallel with CPU.
Warp	Wavefront	Collection of operations that execute in lockstep, run the same instructions, and follow the same control-flow path. Individual lanes can be masked off. Think of this as a vector thread. A 64-wide wavefront is a 64-wide vector op.
Thread block	Workgroup	Group of wavefronts that will be on the GPU at the same time. Can synchronize together and communicate through local memory.
Shared memory	Local memory	Scratchpad that allows communication between wavefronts in a workgroup.
Thread	Work item / Thread	Individual lane in a wavefront. On AMD GPUs, must run in lockstep with other work items in the wavefront. Lanes can be individually masked off. GPU programming models can treat this as a separate thread of execution, though no guarantee of sub-wavefront forward progress.
Local memory	Private memory	Per-thread private memory, often mapped to registers.

Hardware Terminology – Translation key if you are familiar with Nvidia GPUs

Nvidia/CUDA Terminology	AMD Terminology	Description
Streaming Multiprocessor	Compute Unit (CU)	One of many parallel vector processors in a GPU that contain parallel ALUs. All waves in a workgroups are assigned to the same CU.
Shared Memory	Local Data Share (LDS)	Scratchpad RAMs that hold local memory values.
Global Memory	Global Memory	DRAM memory accessible by the GPU that goes through some layers cache
Special Function Unit	N/A	Nvidia GPUs used a special execution pipe for transcendentals and other special intrinsics. AMD GPUs execute them as regular vector instructions.

DISCLAIMER

DISCLAIMER

The information contained herein is for informational purposes only, and is subject to change without notice. While every precaution has been taken in the preparation of this document, it may contain technical inaccuracies, omissions and typographical errors, and AMD is under no obligation to update or otherwise correct this information. Advanced Micro Devices, Inc. makes no representations or warranties with respect to the accuracy or completeness of the contents of this document, and assumes no liability of any kind, including the implied warranties of noninfringement, merchantability or fitness for particular purposes, with respect to the operation or use of AMD hardware, software or other products described herein. No license, including implied or arising by estoppel, to any intellectual property rights is granted by this document. Terms and limitations applicable to the purchase or use of AMD's products are as set forth in a signed agreement between the parties or in AMD's Standard Terms and Conditions of Sale. GD-18